

ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

=====***=====



HaUI

**SCHOOL OF INFORMATION
& COMMUNICATIONS TECHNOLOGY**

BÁO CÁO THỰC NGHIỆM THUỘC HỌC PHẦN:
MỘT SỐ CÔNG NGHỆ PHÁT TRIỂN PHẦN MỀM

ĐỀ TÀI:

NGHIÊN CỨU HỌC MÁY VỚI OPENCV PYTHON VÀ ỨNG DỤNG

GVHD	: TS. Hà Mạnh Đào
Nhóm	: 13
Sinh viên thực hiện	: Nguyễn Anh Tuấn - 2022603876 Ngô Hữu Tùng - 2022604648 Vũ Xuân Bắc - 2022605451
Lớp - Khóa	: 20251IT6024002 - K17

Hà nội - 2025

LỜI CẢM ƠN

Trước hết, chúng em xin gửi lời tri ân chân thành tới thầy Hà Mạnh Đào. Thầy không chỉ là người hướng dẫn mà còn là nguồn động viên và sự hỗ trợ quan trọng trong suốt quá trình học tập và nghiên cứu của chúng em. Sự quan tâm tận tâm và lòng nhiệt huyết của thầy đã giúp chúng em không chỉ thu được kiến thức chuyên ngành mà còn những góc nhìn thực tế quý báu và lời khuyên quý giá.

Dưới sự hướng dẫn của thầy và sự nỗ lực không ngừng của toàn bộ nhóm, chúng em đã kết hợp kiến thức thuần túy từ giảng đường với những kiến thức mới mẻ và kinh nghiệm cá nhân để hoàn thành bài báo cáo. Chúng em trân trọng mọi góp ý, đánh giá chân thành từ thầy cô và các bạn để từ đó hoàn thiện đề tài của chúng em hơn nữa.

Chúng em xin chân thành cảm ơn!

Các sinh viên nhóm 13

MỤC LỤC

LỜI CẢM ƠN	2
DANH MỤC HÌNH ẢNH.....	6
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....	7
1.1 Học máy (Machine Learning) cơ bản.....	7
1.1.1 Khái niệm về học máy	7
1.1.2 Các kiểu hệ thống học máy	7
1.1.3 Ứng dụng của học máy	10
1.2 Tổng quan về thư viện OpenCV	11
1.2.1 Giới thiệu chung và lịch sử phát triển	11
1.2.2 Các tính năng cốt lõi	13
1.3 Cấu trúc hệ thống học máy trong OpenCV	13
1.3.1 Tổng quan về mô-đun học máy (module cv.ml).....	13
1.3.2 Các Mô-đun học máy được cv.ml hỗ trợ.....	14
1.3.3 Quy trình xử lý ảnh tiêu biểu trong ứng dụng học máy	14
CHƯƠNG 2: PHÂN TÍCH MÃ NGUỒN CHƯƠNG TRÌNH.....	16
2.1 Image Blending OpenCV Python.....	16
2.1.1 Mô tả bài toán.....	16
2.1.2 Cấu trúc mạng	16
2.1.3 Đặc điểm dữ liệu huấn luyện	17
2.1.4 Thuật học.....	17
2.1.5 Quá trình huấn luyện	18
2.1.6 Kết quả	18
2.1.7 Chạy và đánh giá	20

2.1.8 Phân tích chi tiết	20
2.2 Handwritten Digit Recognition in Python	25
2.2.1 Mô tả bài toán.....	25
2.2.2 Cấu trúc mạng	25
2.2.3 Đặc điểm dữ liệu huấn luyện	27
2.2.4 Thuật học	27
2.2.5 Quá trình huấn luyện	28
2.2.6 Kết quả	28
2.2.7 Chạy và đánh giá	29
2.2.8 Phân tích chi tiết	30
2.3 Brightness Control with Hand Detection OpenCV Python.....	31
2.3.1 Mô tả bài toán.....	31
2.3.2 Cấu trúc mạng	32
2.3.3 Đặc điểm dữ liệu huấn luyện	32
2.3.4 Thuật học	33
2.3.5 Quá trình huấn luyện	33
2.3.6 Kết quả	33
2.3.7 Chạy và đánh giá	34
2.3.8 Phân tích chi tiết	35
CHƯƠNG 3: ỨNG DỤNG THỰC TẾ	42
3.1 Cài đặt chương trình	42
3.2 Ứng dụng thực tế với “Image Blending OpenCV Python”	45
3.2.1 Tổng quan.....	45
3.2.2 Phân tích mã nguồn chương trình	46
3.2.3 Kết quả và đánh giá	52

TÀI LIỆU THAM KHẢO	56
--------------------------	----

DANH MỤC HÌNH ẢNH

Hình 1.1 Học máy có giám sát.....	7
Hình 1.2 học máy không giám sát	8
Hình 1.3 Học máy bán giám sát.....	9
Hình 1.4 Học tăng cường	10
Hình 2.1. Ảnh 1	19
Hình 2.2. Ảnh 2.....	19
Hình 2.3. Ảnh kết quả	20
Hình 3.1. Giao diện chính.....	53
Hình 3.2. Giao diện chọn ảnh	53
Hình 3.3. Giao diện kết quả trộn ảnh	53
Hình 3.4. Giao diện lưu ảnh	54
Hình 3.5. Giao diện lưu ảnh thành công	54

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

1.1 Học máy (Machine Learning) cơ bản

1.1.1 Khái niệm về học máy

Học máy là một lĩnh vực con của Khoa học máy tính. Nó có khả năng tự học hỏi dựa trên dữ liệu đưa vào mà không cần phải được lập trình cụ thể.

Học máy tập trung vào việc phát triển các chương trình máy tính có thể truy cập dữ liệu và sử dụng nó để tự học.

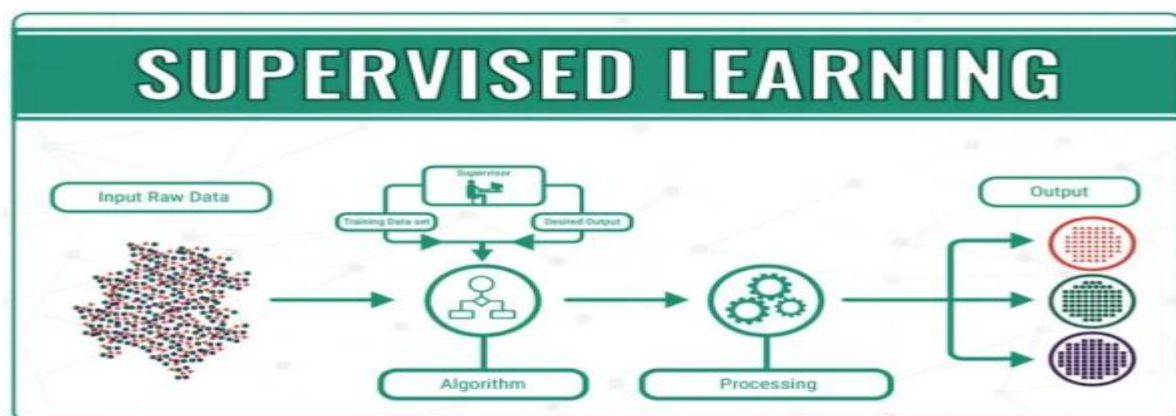
Quá trình học bắt đầu bằng các quan sát hoặc dữ liệu. Ví dụ, để tìm kiếm các mẫu trong dữ liệu và đưa ra quyết định tốt hơn trong tương lai dựa trên các mẫu mà chúng ta cung cấp. Mục đích chính của học máy là cho phép các máy tính tự động học mà không cần sự can thiệp hay trợ giúp của con người và có thể điều chỉnh các hành động tương ứng.

Học máy chính là phương tiện để đạt được mục tiêu của AI, đó là làm cho máy móc có được trí thông minh như con người hoặc có thể hơn con người.

1.1.2 Các kiểu hệ thống học máy

a) Học có giám sát

Machine Learning được giám sát, hay còn được gọi là học máy có giám sát, được định nghĩa bằng cách sử dụng các tập dữ liệu được gắn nhãn để tạo các thuật toán nhằm phân loại dữ liệu hoặc dự đoán kết quả một cách chính xác.

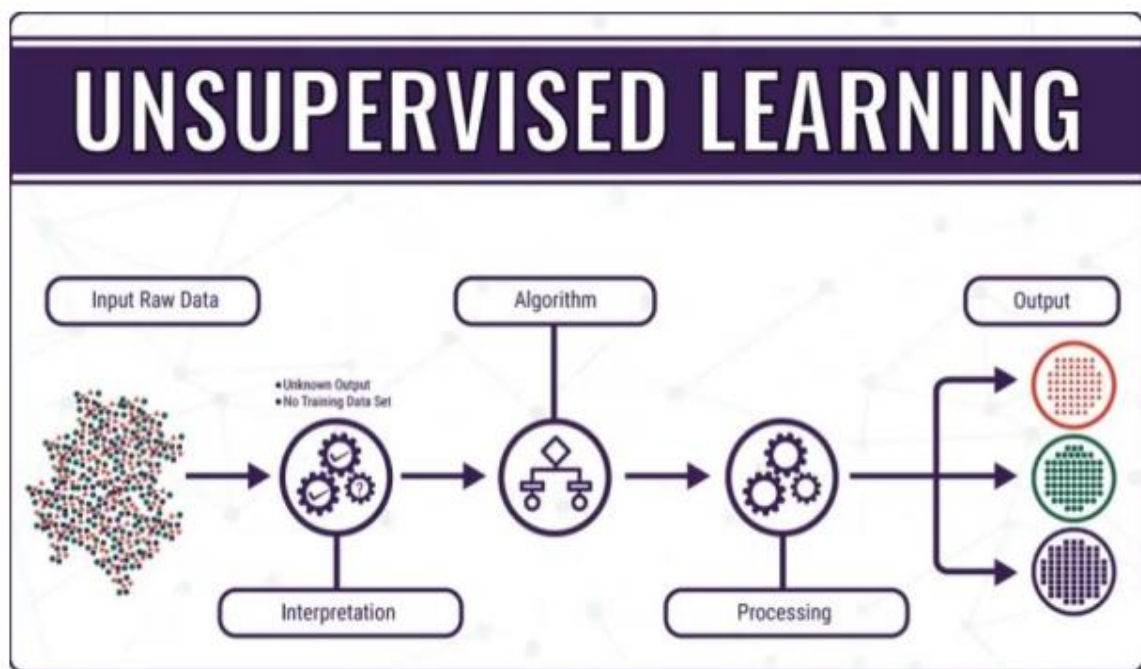


Hình 1.1 Học máy có giám sát

Học máy có giám sát giúp các tổ chức giải quyết nhiều vấn đề trong thế giới thực trên quy mô lớn, chẳng hạn như phân loại thư rác trong một thư mục riêng biệt từ hộp thư đến. Một số phương pháp được sử dụng trong học máy có giám sát bao gồm mạng nơ-ron, Naive Bayes, hồi quy tuyến tính, hồi quy logistic, Random Forest, thuật toán SVM,...

b) Học không giám sát

Machine Learning không giám sát, còn được gọi là học máy không giám sát, sử dụng các thuật toán học máy để phân tích và phân cụm các tập dữ liệu không được gắn nhãn. Các thuật toán này phát hiện ra các mẫu hoặc nhóm dữ liệu ẩn mà không cần sự can thiệp của con người.



Hình 1.2 học máy không giám sát

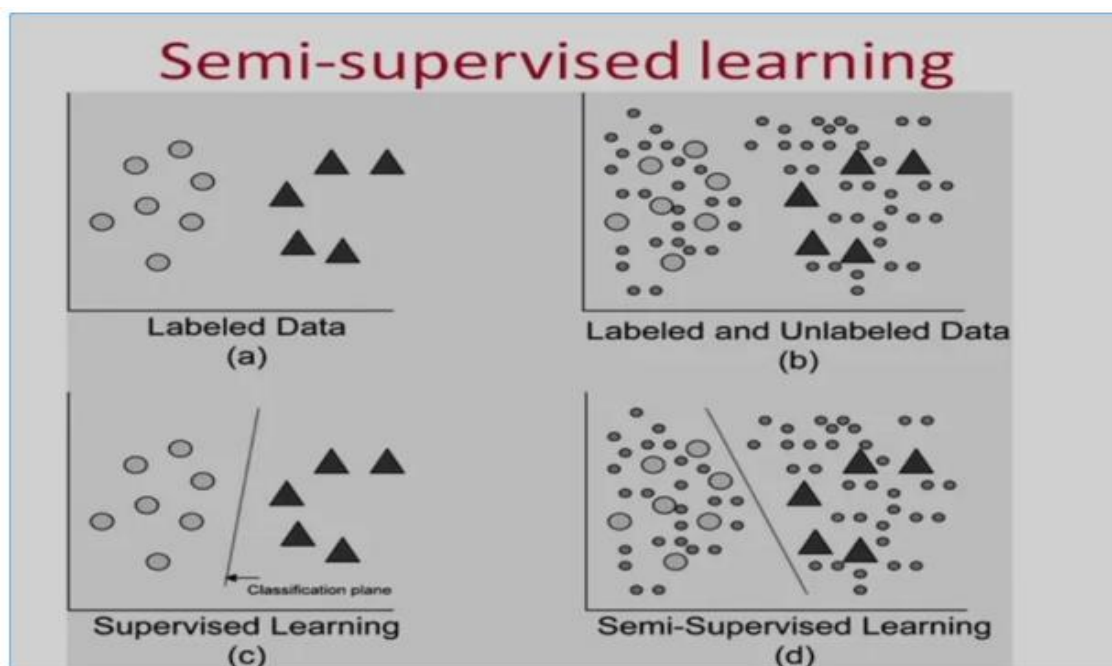
Khả năng phát hiện ra những điểm tương đồng và khác biệt trong thông tin khiến nó trở thành giải pháp lý tưởng để phân tích dữ liệu khám phá, chiến lược bán chéo, phân khúc khách hàng, nhận dạng hình ảnh và mẫu.

Học máy không có giám sát cũng được sử dụng để giảm số lượng tính năng trong một mô hình thông qua quá trình giảm kích thước. Phân tích thành phần chính (PCA) và phân tích giá trị đơn lẻ (SVD) là hai cách tiếp cận phổ biến cho việc này. Các thuật toán khác được sử dụng trong machine learning không giám

sát bao gồm neural network, phân cụm K-mean, phương pháp phân nhóm xác suất,...

c) Học máy bán giám sát

Học máy bán giám sát cung cấp một phương tiện kết nối giữa Machine Learning có giám sát và không giám sát. Trong quá trình đào tạo, nó sử dụng một tập dữ liệu có nhãn nhỏ hơn để hướng dẫn phân loại và trích xuất tính năng từ một tập dữ liệu lớn hơn mà không được gán nhãn. Semi-supervised Learning có thể giải quyết vấn đề không có đủ dữ liệu được gán nhãn để đào tạo thuật toán học có giám sát.



Hình 1.3 Học máy bán giám sát

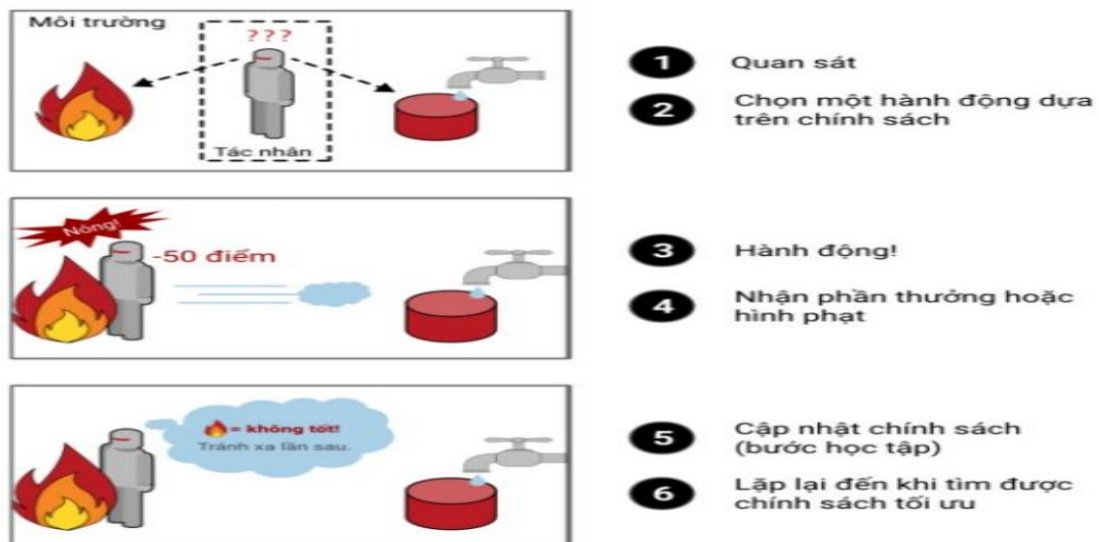
d) Học tăng cường

Học tăng cường là các bài toán giúp cho một hệ thống tự động xác định hành vi dựa trên hoàn cảnh để đạt được lợi ích cao nhất (maximizing the performance). Hiện tại, học tăng cường chủ yếu được áp dụng vào Lý Thuyết Trò Chơi (Game Theory), các thuật toán cần xác định nước đi tiếp theo để đạt được điểm số cao nhất.

Huấn luyện cho máy tính chơi game Mario. Đây là một chương trình thú vị dạy máy tính chơi game Mario. Đầu vào của thuật toán là sơ đồ của màn hình tại thời điểm hiện tại, nhiệm vụ của thuật toán là với đầu vào đó, tổ hợp phím nào

nên được bấm. Việc huấn luyện này được dựa trên điểm số cho việc di chuyển được bao xa trong thời gian bao lâu trong game, càng xa và càng nhanh thì được điểm thưởng càng cao (điểm thưởng này không phải là điểm của trò chơi mà là điểm do chính người lập trình tạo ra). Thông qua huấn luyện, thuật toán sẽ tìm ra một cách tối ưu để tối đa số điểm trên, qua đó đạt được mục đích cuối cùng là cứu công chúa.

Học tăng cường (RL) là phương pháp có cấu trúc rất khác. Hệ thống học trong RL được gọi là tác nhân (agent). Nó có thể quan sát môi trường xung quanh và chọn việc hành động, sau đó nhận về điểm thưởng (reward) hoặc lượng phạt (penalty) dưới dạng điểm thưởng âm, hệ thống sau đó cần tự học chiến lược tốt nhất để nhận về nhiều điểm thưởng nhất qua thời gian, và chiến lược này được gọi là chính sách (policy).



Hình 1.4 Học tăng cường

1.1.3 Ứng dụng của học máy

Nhận diện hình ảnh (Image Recognition): Nhận dạng hình ảnh cũng là một trong những ứng dụng phổ biến của Machine Learning. Machine Learning cũng được ứng dụng để phát hiện khuôn mặt trong ảnh của nhiều người. Có một danh mục riêng cho mỗi người trong cơ sở dữ liệu hình ảnh của nhiều người.

Nhận dạng giọng nói: Hay còn được gọi là nhận dạng giọng nói tự động (ASR – Automatic Speech Recognition), sử dụng Natural Language Processing

(NLP – xử lý ngôn ngữ tự nhiên) để xử lý giọng nói của con người thành định dạng viết. Nhiều thiết bị di động tích hợp tính năng nhận dạng giọng nói vào hệ thống để thực hiện tìm kiếm bằng giọng nói.

Dịch vụ khách hàng: Chatbot trực tuyến đang thay thế các tác nhân con người trong quá trình giao tiếp với khách hàng. Chatbot trả lời các câu hỏi thường gặp (FAQ) xung quanh các chủ đề, như vận chuyển hoặc cung cấp lời khuyên được cá nhân hóa, bán sản phẩm chéo hoặc đề xuất kích thích cho người dùng, thay đổi cách chúng ta nghĩ về mức độ tương tác của khách hàng trên các trang web và nền tảng truyền thông xã hội.

Computer vision: Công nghệ AI này cho phép máy tính và hệ thống lấy thông tin có ý nghĩa từ hình ảnh kỹ thuật số, video và các đầu vào trực quan khác. Dựa trên các đầu vào đó có thể thực hiện hành động. Khả năng cung cấp các khuyến nghị này phân biệt chúng với các nhiệm vụ nhận dạng hình ảnh. Được hỗ trợ bởi mạng nơ-ron phức hợp, computer vision có các ứng dụng trong việc gắn thẻ ảnh trên mạng xã hội, chụp ảnh X quang trong chăm sóc sức khỏe và xe hơi tự lái trong ngành công nghiệp ô tô.

Công cụ đề xuất: Sử dụng dữ liệu hành vi tiêu dùng trong quá khứ, các thuật toán AI có thể giúp khám phá các xu hướng dữ liệu có thể được sử dụng để phát triển các chiến lược bán chéo hiệu quả hơn. Điều này được sử dụng để đưa ra các đề xuất bổ sung có liên quan cho khách hàng trong quá trình thanh toán cho các nhà bán lẻ trực tuyến.

1.2 Tổng quan về thư viện OpenCV

1.2.1 Giới thiệu chung và lịch sử phát triển

OpenCV là tên viết tắt của Opensource Computer Vision library. Là một thư viện nguồn mở cho thị giác máy tính. Cụ thể hơn OpenCV là kho lưu trữ các mã nguồn mở được dùng để xử lý ảnh, phát triển các ứng dụng đồ họa trong thời gian thực.

OpenCV có giao diện thân thiện với mọi ngôn ngữ lập trình như C/C++, Python, Java,... Ngoài ra nó cũng dễ tương thích với các hệ điều hành khác nhau gồm Windows, Linux, Mac OS, IOS và Android.

Lịch sử của thư viện OpenCV (Open Source Computer Vision) bắt đầu từ những năm đầu của thập kỷ 2000 và đã trải qua nhiều phát triển và mở rộng từ đó. Dưới đây là một cái nhìn tổng quan về lịch sử của OpenCV:

- Năm 2000, OpenCV bắt đầu dưới dạng một dự án nghiên cứu tại Intel, được bắt đầu bởi Gary Bradski. Mục tiêu chính của dự án là cung cấp một thư viện mã nguồn mở cho cộng đồng nghiên cứu và phát triển trong lĩnh vực thị giác máy tính.
- Năm 2006, Phiên bản 1.0 của OpenCV được phát hành. Đây là bước quan trọng đánh dấu sự ổn định và chính xác của thư viện, giúp nó trở thành một công cụ phổ biến trong cả cộng đồng nghiên cứu và ngành công nghiệp.
- Năm 2011, OpenCV chính thức trở thành một dự án mã nguồn mở độc lập và không còn thuộc quản lý của Intel. Điều này mở cửa cho sự đóng góp rộng rãi từ cộng đồng phát triển.
- Năm 2012, OpenCV 2.4 được phát hành với nhiều cải tiến và bổ sung, bao gồm cả việc hỗ trợ CUDA cho tăng tốc hiệu suất trên GPU NVIDIA.
- Năm 2015, OpenCV 3.0 được phát hành với nhiều tính năng mới và cải tiến, bao gồm hỗ trợ C++11, module mới như xử lý thị giác máy tính toàn diện (computational photography), và sự đa dạng trong việc hỗ trợ các ngôn ngữ lập trình.
- Năm 2018, OpenCV 4.0 được giới thiệu, với các cải tiến lớn trong hiệu suất và chất lượng. Bản phát hành này bao gồm sự tích hợp với dlib và TensorFlow, cũng như nhiều cập nhật khác nhau.
- Năm 2023, OpenCV tiếp tục phát triển và cập nhật. Nó trở thành một trong những thư viện thị giác máy tính hàng đầu, được sử dụng rộng rãi trong cả nghiên cứu và các dự án ứng dụng thực tế trên toàn thế giới.

Hiện nay, OpenCV không chỉ là một công cụ quan trọng trong nghiên cứu và phát triển, mà còn là một nguồn lực toàn diện và mở rộng cho cộng đồng toàn cầu đang làm việc trong lĩnh vực thị giác máy tính và xử lý ảnh.

1.2.2 Các tính năng cốt lõi

OpenCV cung cấp hơn 2500 thuật toán tối ưu, bao gồm cả những thuật toán cổ điển và hiện đại, giúp giải quyết gần như mọi vấn đề trong lĩnh vực Xử lý Ảnh và Thị giác Máy tính.

OpenCV có rất nhiều tính năng:

- Xử lý ảnh cơ bản như đọc ghi hiển thị ảnh video chuyển đổi màu sắc làm mờ lọc nhiễu phát hiện biên cạnh phân ngưỡng
- Xử lý hình học như xoay ảnh phóng to thu nhỏ dịch chuyển biến đổi affine biến đổi phối cảnh và vẽ các hình học đơn giản
- Theo dõi chuyển động như optical flow tách nền tiền cảnh để theo dõi vật thể di chuyển
- Ứng dụng nâng cao như nhận diện khuôn mặt nhận diện chữ viết OCR xử lý ảnh 3D tạo bản đồ độ sâu ứng dụng AR thực tế tăng cường

OpenCV đóng vai trò là công cụ nền tảng cho các nhà phát triển và nghiên cứu, giúp học dễ dàng triển khai các hệ thống Thị giác Máy tính từ nguyên mẫu đơn giản đến các ứng dụng công nghiệp phức tạp.

1.3 Cấu trúc hệ thống học máy trong OpenCV

1.3.1 Tổng quan về mô-đun học máy (module cv.ml)

Mô-đun học máy là một thành phần quan trọng trong thư viện OpenCV, cung cấp một bộ sưu tập các thuật toán Học máy cổ điển (hoặc "shallow learning") được tối ưu hóa.

Mục đích: Mô-đun học máy được thiết kế để cung cấp các giải pháp Học máy hiệu suất cao, đặc biệt phù hợp cho việc phân loại và hồi quy trên các bộ đặc trưng được trích chọn từ dữ liệu ảnh (ví dụ: các đặc trưng HOG, LBP...).

Ưu điểm: Tích hợp sâu với cấu trúc dữ liệu của OpenCV (như cv.Mat hoặc NumPy array), đảm bảo tốc độ và hiệu quả khi làm việc trực tiếp với dữ liệu ảnh đã qua tiền xử lý. Module này rất hữu ích cho các ứng dụng yêu cầu tốc độ xử lý nhanh, không cần độ phức tạp của các mô hình Học sâu (Deep Learning).

1.3.2 Các Mô-đun học máy được cv.ml hỗ trợ

Module cv.ml bao gồm một loạt các thuật toán phân loại và hồi quy phổ biến:

Mô hình	Thuật toán	Loại bài toán
KNearest	k-NN	Phân loại/Hồi quy
SVM	Support Vector Machine	Phân loại/Hồi quy
DTrees	Decision Trees	Phân loại/Hồi quy
RTrees	Random Trees (RanDom Forest)	Phân loại/Hồi quy
Bost	AdaBoost/Gradient Boosting	Phân loại/Hồi quy
ANN_MLP	Artificial Neural Networks	Phân loại/Hồi quy

1.3.3 Quy trình xử lý ảnh tiêu biểu trong ứng dụng học máy

Hệ thống Thị giác Máy tính sử dụng Học máy cổ điển thường tuân theo một quy trình chuẩn như sau:

1. Thu nhận và tiền xử lý (Acquisition & Pre-processing):
 - Đọc dữ liệu ảnh/video từ nguồn (ví dụ: cv.imread, cv.VideoCapture).
 - Thực hiện các bước tiền xử lý cơ bản để chuẩn hóa dữ liệu: chuyển sang thang độ xám (Grayscale), chuẩn hóa kích thước (Resize), lọc nhiễu (ví dụ: cv.GaussianBlur), và chuẩn hóa màu sắc/cường độ.
2. Trích chọn đặc trưng (Feature Extraction):
 - Bước quan trọng: Chuyển đổi dữ liệu ảnh thô (ma trận điểm ảnh) thành một vector đặc trưng có ý nghĩa, có khả năng phân biệt các lớp (ví dụ: HOG, LBP hoặc các đặc trưng hình dạng, màu sắc).

- Đầu ra là tập hợp các vector đặc trưng và nhãn tương ứng (cho Học có giám sát).

3. Huấn luyện/Suy luận (Training / Inference):

- Huấn luyện: Sử dụng vector đặc trưng để huấn luyện một mô hình Học máy từ module `cv.ml` (ví dụ: `cv.ml.SVM`) để nó học được mối quan hệ giữa đặc trưng và nhãn.
- Suy luận: Với một ảnh mới, trích chọn vector đặc trưng và đưa vào mô hình đã huấn luyện để dự đoán nhãn (phân loại) hoặc giá trị (hồi quy).

4. Hậu xử lý và hiển thị (Post-processing & Visualization):

- Xử lý kết quả dự đoán (ví dụ: vẽ hình chữ nhật bao quanh vật thể được phát hiện).
- Hiển thị kết quả lên màn hình cho người dùng (`cv.imshow`).

CHƯƠNG 2: PHÂN TÍCH MÃ NGUỒN CHƯƠNG TRÌNH

2.1 Image Blending OpenCV Python

2.1.1 Mô tả bài toán

- Tên bài toán: Image Blending OpenCV Python
- Mục tiêu:
 - + Trộn hai ảnh thành một ảnh duy nhất, đảm bảo sự chuyển tiếp mượt mà giữa hai vùng ảnh, không xuất hiện đường ranh cứng.
 - + Giữ lại chi tiết quan trọng của cả hai ảnh ở nhiều mức độ khác nhau (từ chi tiết nhỏ đến tổng thể).
- Đầu vào: Hai ảnh màu (RGB) với kích thước có thể khác nhau (nhưng cần resize để xử lý).
- Đầu ra: Một ảnh duy nhất, đã được kết hợp từ hai ảnh gốc, hiển thị sự chuyển tiếp mượt mà giữa các vùng ảnh.
- Ứng dụng thực tế: Ghép ảnh panorama, overlay, watermark, hoặc tạo hiệu ứng nghệ thuật trong hình ảnh.

2.1.2 Cấu trúc mạng

Mặc dù bài toán không dùng học máy, bạn có thể trình bày thuật toán theo dạng “các tầng xử lý”:

- **Gaussian Pyramid:** Tạo nhiều cấp ảnh nhỏ dần từ ảnh gốc, giúp làm mờ và giảm nhiễu.
- **Laplacian Pyramid:** Tính ảnh chênh lệch giữa các cấp Gaussian liên tiếp để giữ chi tiết quan trọng.
- **Blending từng cấp:** Ghép nửa ảnh 1 với nửa ảnh 2 ở từng cấp Laplacian.
- **Reconstruction:** Dùng `cv.pyrUp` và `cv.add` để tái tạo ảnh cuối cùng từ các Laplacian đã ghép.

Sơ đồ dữ liệu (gợi ý):

Ảnh 1 $\rightarrow$ Gaussian $\rightarrow$ Laplacian \
 $\rightarrow$ Reconstruct $\rightarrow$ Ảnh blend
 Ảnh 2 $\rightarrow$ Gaussian $\rightarrow$ Laplacian /

2.1.3 Đặc điểm dữ liệu huấn luyện

- Hai ảnh màu RGB, có thể khác kích thước và nội dung.
- Nếu kích thước khác nhau, phải resize hoặc pad để các cấp pyramid tương thích.
- Ảnh chất lượng cao (độ phân giải lớn) giúp giữ chi tiết tốt hơn.
- Ảnh đơn giản, rõ ràng, không quá nhiều nền phức tạp để kết quả blend mượt.

2.1.4 Thuật học

- **Gaussian Pyramid:** Giảm kích thước ảnh theo cấp số 2, áp dụng Gaussian filtering.

```
gp_n1[i] = cv.pyrDown(gp_n1[i-1])  
gp_n2[i] = cv.pyrDown(gp_n2[i-1])
```

- **Laplacian Pyramid:** Tính ảnh chênh lệch giữa các cấp Gaussian liên tiếp:
 $L_{i-1} = G_{i-1} - \text{Expand}(G_i)$
 - + **cv.pyrUp(G_i)** mở rộng ảnh cấp thấp hơn để trừ với ảnh cấp trên.
 - + Lưu ý phải resize khớp kích thước trước khi trừ.
- **Blend:** Ghép nửa trái ảnh 1 với nửa phải ảnh 2 ở từng cấp Laplacian (**np.hstack**).
- **Reconstruct:** Tái tạo ảnh cuối cùng từ Laplacian Pyramid đã blend (**cv.pyrUp + cv.add**).

2.1.5 Quá trình huấn luyện

Các bước chi tiết trong chương trình:

1. load 2 ảnh:

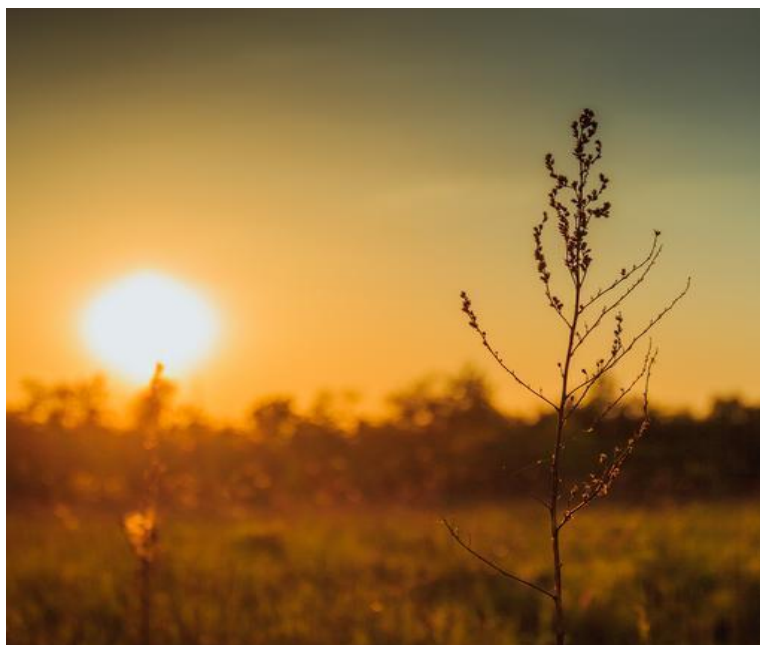
```
n1 = cv.imread("img/nature1.jpg")  
n2 = cv.imread("img/nature2.png")
```

2. Resize nếu cần để ảnh cùng kích thước.
3. Tạo **Gaussian Pyramid** cho từng ảnh (6 cấp).
4. Tạo **Laplacian Pyramid** cho từng ảnh từ Gaussian Pyramid.
5. Ghép nửa ảnh 1 và nửa ảnh 2 ở từng cấp Laplacian.
6. **Reconstruct** ảnh cuối cùng từ các cấp Laplacian đã ghép.
7. Hiển thị hoặc lưu ảnh kết quả:

```
cv.imshow("Blended Image", reconstruct)  
cv.imwrite("blended_output.jpg", reconstruct)
```

2.1.6 Kết quả

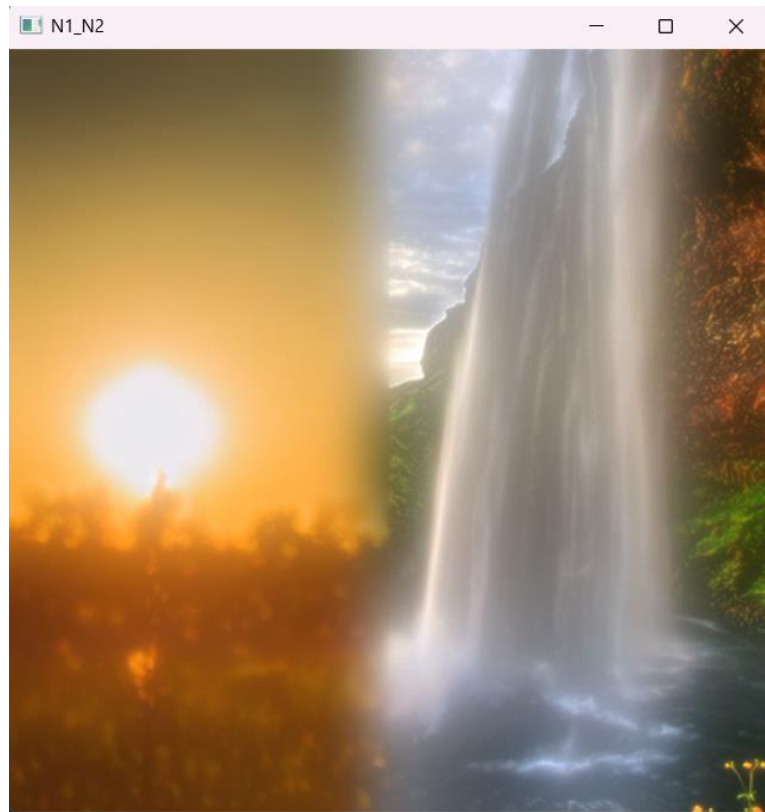
- Ảnh đầu ra hiển thị sự chuyển tiếp mượt mà giữa hai ảnh gốc.
- Các mức pyramid minh họa chi tiết ảnh ở nhiều cấp.
- Có thể thay đổi tỉ lệ ghép (ví dụ $\alpha = 0.3-0.7$) để kiểm soát blend.
- Kết quả đẹp nhất khi ảnh gốc có độ sáng và màu sắc tương đồng.



Hình 2.1. Ảnh 1



Hình 2.2. Ảnh 2



Hình 2.3. Ảnh kết quả

2.1.7 Chạy và đánh giá

- Chạy thử:
 - + Thử với nhiều cặp ảnh khác nhau.
 - + Kiểm tra tốc độ xử lý: ảnh 800x600, xử lý <1 giây.
- Đánh giá:
 - + Chính xác: Các chi tiết từ cả hai ảnh được giữ lại, đường blend mượt.
 - + Tốc độ: Nhanh, gần như tức thời cho ảnh bình thường.
- Giới hạn:
 - + Ảnh có kích thước khác nhau → cần resize hoặc pad.
 - + Ảnh có nền phức tạp → blend có thể không đồng đều.

2.1.8 Phân tích chi tiết

- **Import và docstring**

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt
```

- cv2 (alias cv): thư viện OpenCV cho xử lý ảnh/video.
- numpy: xử lý mảng ma trận.
- matplotlib.pyplot không được dùng trong mã hiện tại — có thể dùng để hiển thị hoặc lưu các pyramid nếu muốn minh họa trong báo cáo.

Docstring ở đầu hàm `blend()` giải thích mục tiêu: tạo Gaussian pyramid, chuyển sang Laplacian pyramid, ghép từng tầng và tái tạo. Viết rõ ràng, đúng yêu cầu báo cáo.

- **Đọc ảnh (I/O)**

```
n1 = cv.imread("img/nature1.jpg")
n2 = cv.imread("img/nature2.png")

print(n1.shape)
print(n2.shape)
```

- `cv.imread(path)` đọc file ảnh trả về `numpy.ndarray` kích thước (height, width, channels) hoặc `None` nếu không tìm thấy/không đọc được.
- Lưu ý bắt buộc: luôn kiểm tra if `n1` is `None` or `n2` is `None`: để tránh `AttributeError` (lỗi thuộc tính).
- `print(...shape)` cho biết kích thước — dùng để debug.

- **Hàng chông ngang thô**

```
# n1_n2 = np.hstack((n1[:, :256], n2[:, 256:]))
```

- Đây là cách ghép thô (simple horizontal stack): sẽ tạo đường nối rõ rệt. Pyramid blending tối thiểu hóa đường nối này.
- Thông tin này dùng để so sánh kết quả np.hstack vs pyramid blend.

- **Tạo Gaussian Pyramid cho ảnh 1**

```
n1_copy = n1.copy()
gp_n1 = [n1_copy]

for i in range(6):
    n1_copy = cv.pyrDown(n1_copy)
    gp_n1.append(n1_copy)
```

- gp_n1 là danh sách các mức Gaussian pyramid; gp_n1[0] = ảnh gốc, gp_n1[1] = ảnh giảm 1 lần ($\frac{1}{2}$ chiều), ..., gp_n1[6] = giảm 6 lần.
- cv.pyrDown(img) giảm kích thước theo tỉ lệ 2 ở mỗi chiều (thường sử dụng gaussian filtering + subsample).
- Tính toán kích thước: nếu ảnh kích thước (H, W), sau k lần: kích thước xấp xỉ ($H/2^k$, $W/2^k$) (làm tròn xuống).

- **Tạo Gaussian Pyramid cho ảnh 2**

```
n2_copy = n2.copy()
gp_n2 = [n2_copy]

for i in range(6):
    n2_copy = cv.pyrDown(n2_copy)
    gp_n2.append(n2_copy)
```

- gp_n2 là danh sách các mức Gaussian pyramid; gp_n2[0] = ảnh gốc, gp_n2[1] = ảnh giảm 1 lần ($\frac{1}{2}$ chiều), ..., gp_n2[6] = giảm 6 lần.
- cv.pyrDown(img) giảm kích thước theo tỉ lệ 2 ở mỗi chiều (thường sử dụng Gaussian filtering + subsample).

- Tính toán kích thước: nếu ảnh kích thước (H, W), sau k lần: kích thước xấp xỉ ($H/2^k$, $W/2^k$) (làm tròn xuống).

- **Tạo Laplacian Pyramid cho ảnh 1**

```
n1_copy = gp_n1[5]
lp_n1 = [n1_copy]

for i in range(5, 0, -1):
    gp_ex = cv.pyrUp(gp_n1[i])
    lap = cv.subtract(gp_n1[i-1], gp_ex)
    lp_n1.append(lap)
```

- Công thức: $L_{i-1} = G_{i-1} - \text{Expand}(G_i)$, trong đó $\text{pyrUp}(G_i)$ là phép mở rộng (interpolation) để đưa G_i về kích thước G_{i-1} .
- `lp_n1` lưu Laplacian levels: `lp_n1[0]` cấp nhỏ nhất (ở code lấy `gp_n1[5]` làm phần tử đầu)
- Quan trọng: nếu `pyrUp(gp_n1[i])` có kích thước khác `gp_n1[i-1]` sẽ gây lỗi `subtract`. Phải đảm bảo kích thước khớp (`resize/pad`).

- **Tạo Laplacian Pyramid cho ảnh 2**

```
n2_copy = gp_n2[5]
lp_n2 = [n2_copy]

for i in range(5, 0, -1):
    gp_ex = cv.pyrUp(gp_n2[i])
    lap = cv.subtract(gp_n2[i-1], gp_ex)
    lp_n2.append(lap)
```

- Công thức: $L_{i-1} = G_{i-1} - \text{Expand}(G_i)$, trong đó $\text{pyrUp}(G_i)$ là phép mở rộng (interpolation) để đưa G_i về kích thước G_{i-1} .
- `lp_n2` lưu Laplacian levels: `lp_n2[0]` cấp nhỏ nhất (ở code lấy `gp_n2[5]` làm phần tử đầu — cần giải thích ordering trong báo cáo).
- Quan trọng: nếu `pyrUp(gp_n2[i])` có kích thước khác `gp_n2[i-1]` sẽ gây lỗi `subtract` (là lỗi bạn đã gặp trước đó).

- Ghép nửa trái — nửa phải ở mỗi level

```
n1_n2_pyramid = []
n = 0
for n1_lap, n2_lap in zip(lp_n1, lp_n2):
    n += 1
    cols, rows, ch = n1_lap.shape
    laplacian = np.hstack(
        (n1_lap[:, 0:int(cols/2)], n2_lap[:, int(cols/2):]))
    n1_n2_pyramid.append(laplacian)
```

- Mục tiêu: cho mỗi mức Laplacian, cắt nửa trái của ảnh 1, nửa phải của ảnh 2 rồi ghép ngang.
- BUG/nhầm biến: cols, rows, ch = n1_lap.shape là sai tên biến vì shape trả về (height,width,channels). Đặt h,w,ch = n1_lap.shape sẽ rõ ràng hơn. Dùng w//2 chứ không cols/2.
- np.hstack((left,right)) yêu cầu left và right cùng chiều cao; nếu không sẽ gây ValueError: dimensions must match — nguyên nhân phổ biến khi hai ảnh lúc này khác kích thước.

- Tái tạo ảnh cuối (reconstruction)

```
reconstruct = n1_n2_pyramid[0]
for i in range(1, 6):
    reconstruct = cv.pyrUp(reconstruct)
    reconstruct = cv.add(n1_n2_pyramid[i], reconstruct)
```

- Khởi tạo từ cấp nhỏ nhất rồi lần lượt pyrUp và cộng Laplacian tương ứng để tái tạo lên.
- cv.add làm phép cộng ma trận (với clipping lên 255 nếu overflow).
- Lưu ý kích thước: sau pyrUp có thể cần resize để khớp shape của n1_n2_pyramid[i].

- Hiển thị kết quả


```
cv.imshow("N1_N2", reconstruct)
cv.waitKey()
cv.destroyAllWindows()
```

- Mở cửa sổ hiển thị ảnh (yêu cầu chạy trên môi trường có GUI).
- `cv.waitKey()` đợi phím bấm; nếu muốn auto close sau N ms thì truyền số mili giây.
- Để lưu kết quả dùng `cv.imwrite("out.jpg", reconstruct)`.

2.2 Handwritten Digit Recognition in Python

2.2.1 Mô tả bài toán

Bài toán Nhận dạng chữ số viết tay (Handwritten Digit Recognition) là khả năng để máy tính xác định đúng chữ số do con người viết từ một hình ảnh đầu vào. Đây là một bài toán khó do chữ viết tay rất đa dạng, thiếu đồng nhất về kích thước, độ đậm nét và phong cách viết.

Bài toán được phát biểu dưới dạng **phân loại đa lớp**, trong đó hệ thống cần phân loại một hình ảnh chữ số viết tay vào một trong **10 lớp rời rạc (0–9)**.

Đầu vào của mô hình là ảnh thang độ xám kích thước **28×28 pixel**, và **đầu ra** là nhãn dự đoán kèm **độ tin cậy (Confidence Score)**.

Tập dữ liệu **MNIST** gồm **70.000 ảnh**, và áp dụng **Mạng Nơ-ron Tích chập (Convolutional Neural Network - CNN)** để tự động trích xuất đặc trưng và nhận diện chữ số. Đồng thời, một **giao diện người dùng (GUI)** được xây dựng cho phép người dùng vẽ trực tiếp chữ số và nhận kết quả tức thì.

2.2.2 Cấu trúc mạng

Mạng CNN được thiết kế với các lớp như sau:

- **Conv2D (32 bộ lọc, kernel 5x5, ReLU):**
 - Thực hiện phép tích chập với 32 bộ lọc kích thước 5x5

- **Mục đích:** trích xuất các đặc trưng cục bộ (edges, strokes) của chữ số.
- **MaxPooling2D (2x2):**
 - Giảm kích thước không gian (downsampling) bằng cách lấy giá trị lớn nhất trong vùng 2×2 .
 - **Mục đích:** giảm số tham số, giữ đặc trưng quan trọng, chống overfitting.
- **Conv2D (64 bộ lọc, kernel 3x3, ReLU):**
 - Tăng số bộ lọc để thu được nhiều đặc trưng phức tạp hơn.
 - **Mục đích:** học các đặc trưng sâu hơn (cấu trúc hình dạng số).
- **MaxPooling2D (2x2):** Tiếp tục giảm kích thước, tạo biểu diễn đặc trưng gọn hơn.
- **Flatten:**
 - Chuyển ma trận đặc trưng 2D sau pooling thành vector 1 chiều.
 - **Mục đích:** chuẩn bị dữ liệu cho các lớp fully connected.
- **Dense 128 + Dropout 0.3:**
 - Lớp fully connected học quan hệ phi tuyến giữa các đặc trưng.
 - Dropout 0.3 bỏ ngẫu nhiên 30% neuron trong quá trình học.
 - **Mục đích:** giảm overfitting, tăng khả năng tổng quát.
- **Dense 64 + Dropout 0.5:**
 - Tiếp tục học biểu diễn đặc trưng.
 - Dropout 0.5 mạnh hơn giúp chống overfitting hiệu quả hơn.
- **Dense 10 + Softmax:**
 - Lớp đầu ra với 10 neuron tương ứng 10 chữ số (0–9).
 - **Softmax:** chuẩn hóa thành 10 xác suất, chọn lớp có xác suất cao nhất.

```

model = Sequential([
    Conv2D(filters: 32, kernel_size: (5,5), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D(pool_size: 2, strides: 2),
    Conv2D(filters: 64, kernel_size: (3,3), activation='relu'),
    MaxPooling2D(pool_size: 2, strides: 2),
    Flatten(),
    Dense(units: 128, activation='relu'),
    Dropout(0.3),
    Dense(units: 64, activation='relu'),
    Dropout(0.5),
    Dense(units: 10, activation='softmax')
])

```

2.2.3 Đặc điểm dữ liệu huấn luyện

Tập dữ liệu sử dụng là **MNIST**, một bộ dữ liệu chuẩn cho bài toán nhận dạng chữ số viết tay.

Kích thước:

- **Tập huấn luyện:** 60.000 ảnh
- **Tập kiểm tra:** 10.000 ảnh

Đặc điểm:

- Mỗi ảnh có kích thước 28x28 pixel, dạng thang độ xám (1 kênh màu)
- Nhãn là các chữ số từ 0 đến 9
- Các nhãn được mã hóa One-Hot Encoding, ví dụ:

$$3 \rightarrow [0,0,0,1,0,0,0,0,0,0]$$

2.2.4 Thuật học

Mô hình sử dụng **Convolutional Neural Network (CNN)** - một loại mạng học sâu hiệu quả cho xử lý ảnh nhờ khả năng tự động trích xuất đặc trưng.

- Hàm mất mát (Loss Function): categorical_crossentropy - hàm tiêu chuẩn cho các bài toán phân loại đa lớp khi sử dụng One-Hot Encoding cho nhãn.
- Bộ tối ưu hóa (Optimizer): adam - thuật toán tối ưu phổ biến, điều chỉnh tốc độ học cho từng tham số của mạng một cách thích nghi.
- Chỉ số đánh giá (Metrics): accuracy - đo tỷ lệ dự đoán đúng trên tổng mẫu.

2.2.5 Quá trình huấn luyện

Mô hình sử dụng phương pháp **Học có Giám sát (Supervised Learning)**, trong đó mỗi ảnh đầu vào đi kèm nhãn tương ứng (0-9).

- **Biên dịch mô hình:** xác định hàm mất mát, bộ tối ưu hóa và chỉ số đánh giá:

```
model.compile(loss="categorical_crossentropy",  
              optimizer="adam",  
              metrics=["accuracy"])
```

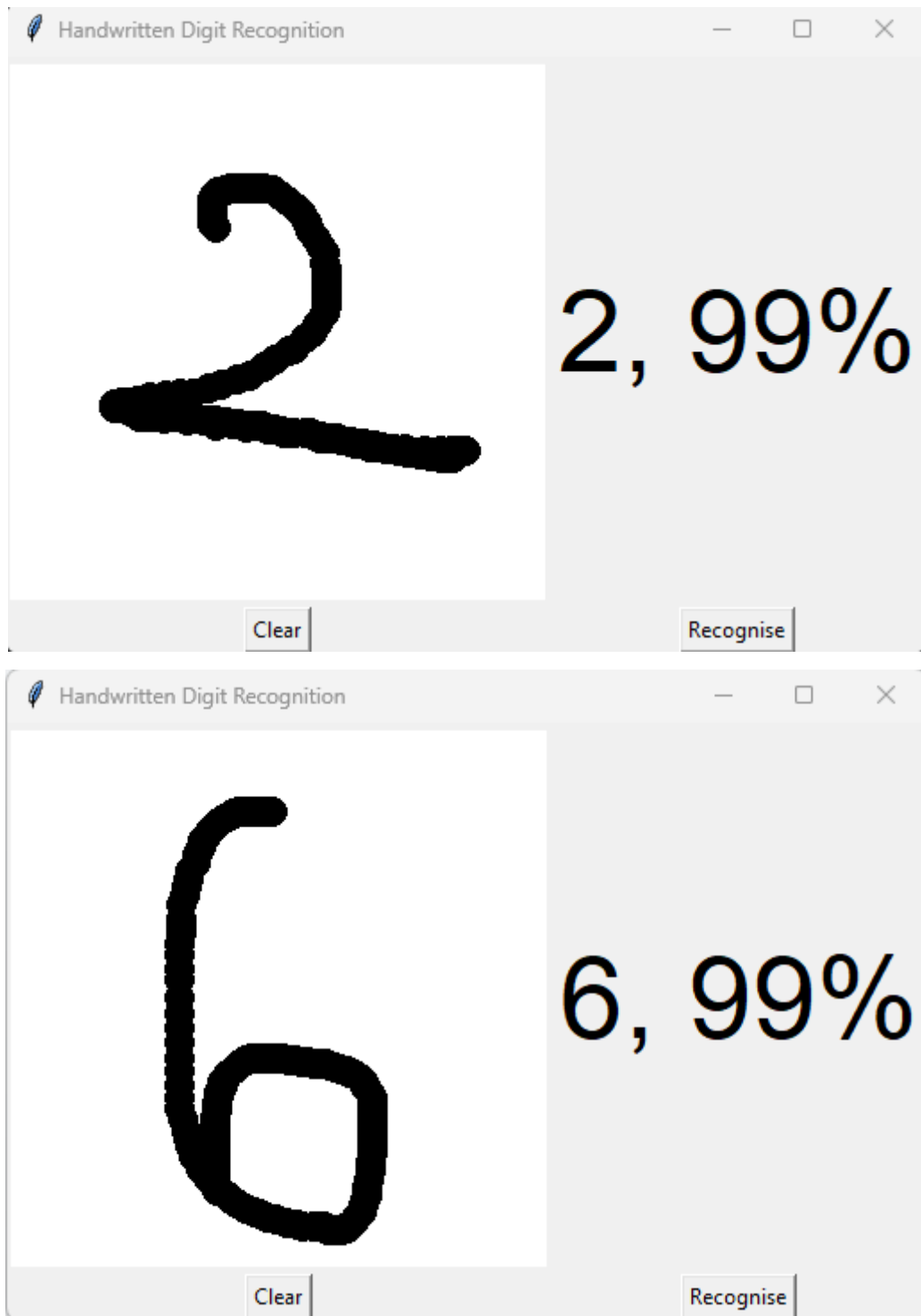
- **Huấn luyện mô hình (Fit):** mô hình học tập từ tập dữ liệu huấn luyện thông qua thuật toán lan truyền ngược:
 - “epochs = 5”: dữ liệu được lặp qua toàn bộ 5 lần.
 - “batch_size = 128”: cập nhật trọng số mỗi khi xử lý 128 mẫu.
 - “validation_data = (x_test, y_test)”: dùng tập kiểm tra để đánh giá sau mỗi epoch (không tham gia huấn luyện).

```
model.fit(x_train, y_train, epochs=5, batch_size=128,  
         validation_data=(x_test, y_test))
```

- Lưu mô hình huấn luyện: `model.save("mnist.h5")`

2.2.6 Kết quả

Mô hình đạt ~**98-99%** trên tập kiểm tra. Mạng đủ chính xác để nhận diện các chữ số viết tay với độ tin cậy cao, đồng thời có thể lưu lại và sử dụng trực tiếp mà không cần huấn luyện lại.



2.2.7 Chạy và đánh giá

Sau khi mô hình CNN được huấn luyện và lưu lại dưới dạng tệp mnist.h5, hệ thống được tích hợp vào một giao diện người dùng (GUI) phát triển bằng thư viện **Tkinter**. Giao diện cho phép người dùng vẽ trực tiếp chữ số viết tay trên một khung canvas. Khi nhấn nút **Recognise**, chương trình sẽ thực hiện chuỗi xử lý sau để đưa ảnh vào mô hình:

1. Chụp nội dung canvas bằng **ImageGrab** dựa trên vị trí thật của cửa sổ Tkinter.
2. Chuyển ảnh sang grayscale để phù hợp định dạng MNIST.
3. Đảo màu (invert).
4. Resize về 28x28 pixel - đúng kích thước yêu cầu của mô hình.
5. Chuẩn hóa pixel về [0,1] và reshape thành tensor (1,28,28,1).
6. Dự đoán bằng mô hình CNN, lấy:
 - nhãn dự đoán (argmax)
 - xác suất dự đoán cao nhất (max)
7. Kết quả được hiển thị: **<chữ số>, <độ chính xác>%**

Đánh giá cho thấy mô hình nhận diện tốt hầu hết các chữ số, đặc biệt với những **nét viết rõ ràng**. Độ tin cậy xuất ra (ví dụ 99%) phản ánh phân phối xác suất của mô hình ở lớp **Softmax**. Nhờ kiến trúc CNN và tập dữ liệu MNIST chuẩn hóa, hệ thống hoạt động ổn định và có thể nhận dạng chính xác ngay cả khi người dùng viết nét đậm, nét mảnh hoặc không hoàn toàn cân đối.

2.2.8 Phân tích chi tiết

a. Phần xây dựng và huấn luyện mô hình

- Sử dụng CNN: việc sử dụng các lớp Conv2D và MaxPooling2D cho phép mô hình tự động học các đặc trưng tối ưu (như cạnh, đường cong) trực tiếp từ pixel, loại bỏ nhu cầu trích chọn đặc trưng thủ công.
- Chuẩn hóa và mã hóa: chuẩn hóa pixel về [0,1] và sử dụng One-Hot Encoding cho nhãn là các bước tiền xử lý bắt buộc để tối ưu hóa việc huấn luyện mạng nơ-ron.
- Dropout: Việc thêm các lớp Dropout (0.3% và 0.5%) là một kỹ thuật chuẩn hóa quan trọng, giúp mạng không học thuộc lòng dữ liệu huấn luyện.

b. Phần GUI (Tkinter) và suy luận

- Tiền xử lý ảnh trực tiếp (predict_digit): Phần quan trọng nhất trong việc kết nối GUI với mô hình

- Đảo ngược màu: Bắt buộc phải đảo ngược màu (trắng → đen, đen → trắng) vì MNIST sử dụng chữ trắng trên nền đen (hoặc chữ đen trên nền trắng với giá trị pixel nhỏ hơn 1), trong khi Canvas sử dụng nét đen trên nền trắng.
- Chuyển đổi dữ liệu: Ảnh PIL (dữ liệu 8 bit) phải được chuyển thành NumPy array (float32) và reshape về định dạng (1,28,28,1) để phù hợp với yêu cầu đầu vào của mô hình Keras CNN.
- Mô hình Tkinter: Lớp App sử dụng các widget cơ bản (Canvas, Label, Button) để tạo giao diện người dùng, sử dụng sự kiện B1-Motion để vẽ và win32gui/ImageGrab để chụp lại vùng vẽ, đây là cách tiếp cận phổ biến khi kết hợp GUI Python tiêu chuẩn với xử lý ảnh.

2.3 Brightness Control with Hand Detection OpenCV Python

2.3.1 Mô tả bài toán

- Tên bài toán: Brightness Control With Hand Detection OpenCV Python
- Mục tiêu: Xây dựng một ứng dụng sử dụng camera để phát hiện cử chỉ tay (cụ thể là khoảng cách giữa ngón cái và ngón trỏ) và sử dụng khoảng cách đó để điều chỉnh độ sáng màn hình của máy tính theo thời gian thực.
- Đầu vào:
 - + Dữ liệu: Dữ liệu video/hình ảnh trực tiếp từ Webcam.
 - + Thư viện/Công cụ: OpenCV (xử lý hình ảnh), Mediapipe (phát hiện landmark tay), screen_brightness_control (điều chỉnh độ sáng hệ thống).
- Đầu ra:
 - + Xử lý: Tọa độ các điểm mốc (landmarks) của bàn tay (cụ thể là ngón cái và ngón trỏ).
 - + Kết quả: Giá trị độ sáng màn hình (từ 0 đến 100) được cập nhật.
- Ứng dụng thực tế:
 - + Tương tác tự nhiên, không chạm (contactless) với máy tính.

- + Hệ thống điều khiển thông minh trong các môi trường đặc biệt (ví dụ: phòng sạch, phòng thí nghiệm).
- + Cải thiện trải nghiệm người dùng với các giao diện người-máy (HMI) mới.

2.3.2 Cấu trúc mạng

Dự án này chủ yếu sử dụng các **thư viện Thị giác Máy tính (Computer Vision)**, không phải là một mô hình mạng nơ-ron tùy chỉnh được huấn luyện từ đầu.

Công nghệ chính: Sử dụng **MediaPipe Hands** (một framework của Google) để phát hiện và theo dõi các **21 điểm mốc (landmarks)** của bàn tay trong thời gian thực.

Các bước xử lý:

1. **Thu thập hình ảnh:** `cv2.VideoCapture(0)`.
2. **Tiền xử lý & Phát hiện:** Chuyển đổi màu sắc (BGR sang RGB) và gọi hàm `hands.process(imgRGB)` của **Mediapipe** để phát hiện tay và landmarks.
3. **Xác định điểm mốc:** Truy xuất tọa độ của **đầu ngón cái (landmark ID 4)** và **đầu ngón trỏ (landmark ID 8)**.
4. **Tính khoảng cách:** Sử dụng định lý **Pythagoras** (hàm `hypot`) để tính khoảng cách Euclid giữa hai điểm mốc này: $\text{length} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.
5. **Ánh xạ (Mapping):** Sử dụng hàm **nội suy tuyến tính** (`np.interp`) để ánh xạ khoảng cách pixel tính được sang dải giá trị độ sáng màn hình (0-100).

2.3.3 Đặc điểm dữ liệu huấn luyện

(Dự án này không yêu cầu huấn luyện dữ liệu tùy chỉnh)

- Nó sử dụng một mô hình **Deep Learning** đã được **huấn luyện trước** (pre-trained model) từ **MediaPipe Hands** để thực hiện việc phát hiện tay và landmarks.

- **Dữ liệu huấn luyện của Mediapipe Hands** thường bao gồm một tập hợp lớn các hình ảnh tay với các tư thế, kích cỡ, và điều kiện ánh sáng khác nhau để đảm bảo tính mạnh mẽ và chính xác khi phát hiện 21 landmarks.

2.3.4 Thuật học

Phát hiện tay (Hand Detection): Sử dụng mô hình **MediaPipe Hands** (thường là sự kết hợp của một mô hình phát hiện lòng bàn tay và một mô hình định vị 21 landmarks).

Tính toán khoảng cách: Thuật toán **Khoảng cách Euclide** (Euclidean Distance).

Ánh xạ giá trị (Value Mapping): Thuật toán **Nội suy Tuyến tính** (Linear Interpolation), được thực hiện bằng hàm `np.interp()`.

- Cụ thể: Ánh xạ khoảng cách `length` (trong dải `[15, 220]` pixel) sang giá trị độ sáng `bright` (trong dải `[0, 100]` phần trăm).
- Công thức ánh xạ (tổng quát):

$$bright = old_min + \frac{(old_value - new_min) \times (old_max - old_min)}{(new_max - new_min)}$$

2.3.5 Quá trình huấn luyện

Không có quá trình huấn luyện trong quá trình thực thi của người dùng, vì nó sử dụng mô hình đã huấn luyện trước.

Dự án này là một ứng dụng suy luận (Inference) trong thời gian thực.

2.3.6 Kết quả

- Hệ thống có thể phát hiện bàn tay và vẽ các đường nối landmarks.
- Một đường kẻ và các vòng tròn được vẽ giữa đầu ngón cái và ngón trỏ.
- Giá trị **độ sáng (bright)** được tính toán và **in ra console**, đồng thời **áp dụng** cho độ sáng hệ thống.

2.3.7 Chạy và đánh giá

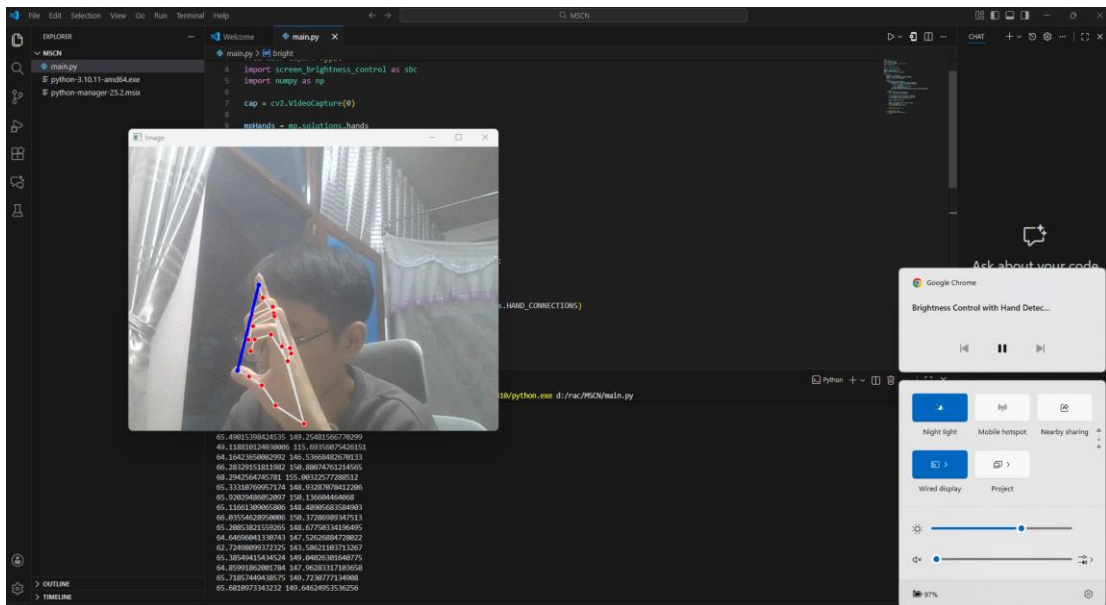
- Chạy thử:

Yêu cầu: Đảm bảo cài đặt các thư viện cần thiết (cv2, mediapipe, screen_brightness_control, numpy).

Thao tác:

1. Mở webcam bằng cv2.VideoCapture(0).
2. Đưa bàn tay lên trước camera.
3. Thay đổi khoảng cách giữa đầu ngón cái (4) và đầu ngón trỏ (8).
 - + Khi ngón tay khép lại (khoảng cách \approx 15), độ sáng sẽ về 0%.
 - + Khi ngón tay mở rộng tối đa (khoảng cách \approx 220), độ sáng sẽ lên 100%.
 - + Cử động các ngón tay khác không ảnh hưởng đến độ sáng.

Hình ảnh kết quả:



- Đánh giá:

Ưu điểm: Đơn giản và hiệu quả: Sử dụng mô hình Mediapipe đã tối ưu hóa, hoạt động nhanh.

- **Tương tác trực quan:** Cử chỉ tay mở rộng/thu hẹp rất tự nhiên để điều khiển độ sáng.

Hạn chế/Giới hạn:

- **Phụ thuộc vào ánh sáng:** Hiệu suất có thể giảm trong điều kiện ánh sáng yếu hoặc quá sáng do việc phát hiện tay bị ảnh hưởng.
- **Phạm vi cố định:** Dải khoảng cách $\left[15, 220\right]$ là cố định và có thể cần **hiệu chỉnh** lại tùy thuộc vào kích thước tay của người dùng và khoảng cách từ tay đến camera.
- **Độ ổn định của screen_brightness_control:** Thư viện này có thể gặp vấn đề về quyền truy cập hoặc không tương thích trên một số hệ điều hành hoặc máy tính cá nhân.
- **Tốn tài nguyên:** Việc xử lý video liên tục có thể tiêu tốn CPU/GPU.

2.3.8 Phân tích chi tiết

a) Khai báo Thư viện và Khởi tạo (Imports and Initialization)

Dòng Mã	Mục đích	Chi tiết
import cv2	OpenCV	Thư viện chính cho xử lý hình ảnh và video.
import mediapipe as mp	MediaPipe	Thư viện của Google để phát hiện và theo dõi các điểm mốc (landmarks) của tay.
from math import hypot	Tính toán	Nhập hàm hypot để tính độ dài cạnh huyền (khoảng cách Euclid) giữa hai điểm.
import screen_brightn ess_control as sbc	Điều khiển Hệ thống	Thư viện để điều chỉnh độ sáng màn hình của hệ điều hành.

<code>import numpy as np</code>	Xử lý số	Thư viện để thực hiện các phép toán mảng, đặc biệt là hàm interp (nội suy tuyến tính).
<code>cap = cv2.VideoCapture(0)</code>	Camera	Khởi tạo đối tượng VideoCapture để truy cập webcam (thường là camera mặc định số 0).
<code>mpHands = mp.solutions.hands</code>	Giải pháp Hands	Truy cập mô-đun chứa các giải pháp cho việc phát hiện tay trong MediaPipe.
<code>hands = mpHands.Hands()</code>	Mô hình Hand	Khởi tạo đối tượng Hands. Đây là mô hình Deep Learning thực hiện việc phát hiện tay và landmarks.
<code>mpDraw = mp.solutions.drawing_utils</code>	Hỗ trợ Vẽ	Đối tượng chứa các hàm tiện ích để vẽ các kết nối và điểm mốc trên bàn tay.

b) Vòng lặp Chính và Xử lý Video (Main Loop and Video Processing)

while True:

 success,img = cap.read()

 imgRGB = cv2.cvtColor(img,cv2.COLOR_BGR2RGB)

 results = hands.process(imgRGB)

Dòng Mã	Mô tả
while True:	Bắt đầu vòng lặp vô hạn để xử lý các khung hình video liên tục.

success,img = cap.read()	Đọc một khung hình (img) từ camera. success là biến boolean cho biết việc đọc có thành công không.
imgRGB = cv2.cvtColor(img,cv2.COLOR_BGR2RGB)	Chuyển đổi khung hình từ không gian màu BGR (mặc định của OpenCV) sang RGB . MediaPipe hoạt động tốt nhất trên định dạng RGB.
results = hands.process(imgRGB)	Xử lý hình ảnh RGB bằng mô hình MediaPipe Hands để tìm kiếm bàn tay và các điểm mốc. Kết quả được lưu trong biến results.

c) Phát hiện và Trích xuất Điểm mốc (Landmark Detection and Extraction)

```
lmList = []
```

```
if results.multi_hand_landmarks:
```

```
    for handlandmark in results.multi_hand_landmarks:
```

```
        for id,lm in enumerate(handlandmark.landmark):
```

```
            h,w,_ = img.shape
```

```
            cx,cy = int(lm.x*w),int(lm.y*h)
```

```
            lmList.append([id,cx,cy])
```

```
mpDraw.draw_landmarks(img,handlandmark,mpHands.HAND_CONNECTIONS)
```

Dòng Mã	Mô tả
lmList = []	Khởi tạo danh sách trống để lưu trữ các điểm mốc (Landmarks List). Mỗi phần tử sẽ là [id, cx, cy].

if results.multi_hand_landmarks:	Kiểm tra xem có bàn tay nào được phát hiện trong khung hình hay không.
for handlandmark in ...:	Lặp qua từng bàn tay được phát hiện (hỗ trợ nhiều bàn tay).
for id,lm in enumerate(...):	Lặp qua 21 điểm mốc của mỗi bàn tay. id là chỉ số (0-20), lm là tọa độ chuẩn hóa (từ 0.0 đến 1.0).
h,w,_ = img.shape	Lấy chiều cao (h) và chiều rộng (w) của khung hình.
cx,cy = int(lm.x*w),int(lm.y *h)	Chuẩn hóa ngược: Chuyển đổi tọa độ chuẩn hóa (lm.x, lm.y) thành tọa độ pixel thực tế (cx, cy) trên khung hình.
lmList.append([id,c x,cy])	Lưu trữ ID và tọa độ pixel của điểm mốc vào danh sách lmList.
mpDraw.draw_land marks(...)	Sử dụng MediaPipe Drawing Utils để vẽ các điểm mốc và các đường kết nối lên khung hình (img).

d) Logic Điều khiển Độ sáng (Brightness Control Logic)

if lmList != []:

 x1,y1 = lmList[4][1],lmList[4][2]

 x2,y2 = lmList[8][1],lmList[8][2]

 cv2.circle(img,(x1,y1),4,(255,0,0),cv2.FILLED)

 # ... (Vẽ các điểm khác)

 cv2.line(img,(x1,y1),(x2,y2),(255,0,0),3)

```
length = hypot(x2-x1,y2-y1)
```

```
bright = np.interp(length,[15,220],[0,100])
```

```
print(bright,length)
```

```
sbc.set_brightness(int(bright))
```

Dòng Mã	Mô tả
if lmList != []:	Chỉ thực hiện logic điều khiển nếu có ít nhất một bàn tay được phát hiện.
x1,y1 = lmList[4][1],lmList[4][2]	Lấy tọa độ X, Y của đầu ngón cái (Landmark ID 4).
x2,y2 = lmList[8][1],lmList[8][2]	Lấy tọa độ X, Y của đầu ngón trỏ (Landmark ID 8).
cv2.circle(...) và cv2.line(...)	Vẽ một đường kẻ và hai hình tròn màu xanh (Blue BGR: 255,0,0) nối giữa đầu ngón cái và ngón trỏ để cung cấp phản hồi hình ảnh.
length = hypot(x2-x1,y2-y1)	Tính khoảng cách Euclid (tính bằng pixel) giữa hai điểm mốc (ngón cái và ngón trỏ).

<pre>bright = np.interp(length,[15,220],[0,100])</pre>	Ảnh xạ tuyến tính (Linear Interpolation): Đây là bước quan trọng nhất.
	* Khoảng cách đầu vào: Nếu length nằm trong dải $\left[15, 220\right]$ pixel.
	* Giá trị đầu ra: Ảnh xạ tuyến tính length sang dải độ sáng $\left[0, 100\right]$ phần trăm. Ví dụ:
	> Nếu $length = 15 \rightarrow bright = 0$.
	> Nếu $length = 220 \rightarrow bright = 100$.
<pre>print(bright,length)</pre>	In giá trị độ sáng tính toán và khoảng cách ra console để theo dõi.
<pre>sbc.set_brightness(int(bright))</pre>	Áp dụng độ sáng: Đặt độ sáng màn hình hệ thống bằng giá trị bright đã tính toán (chuyển sang số nguyên).

e) **Hiển thị và Kết thúc (Display and Exit)**

```
cv2.imshow('Image',img)
```

```
    if cv2.waitKey(1) & 0xff==ord('q'):
```

```
        break
```

Dòng Mã	Mô tả
cv2.imshow('Image',img)	Hiển thị khung hình video (img) đã được xử lý (có vẽ landmarks) trong một cửa sổ tên là 'Image'.
if cv2.waitKey(1) & 0xff==ord('q'):	Chờ 1 mili giây cho một sự kiện bàn phím. Nếu phím 'q' được nhấn, vòng lặp while sẽ kết thúc (break).

CHƯƠNG 3: ỨNG DỤNG THỰC TẾ

3.1 Cài đặt chương trình

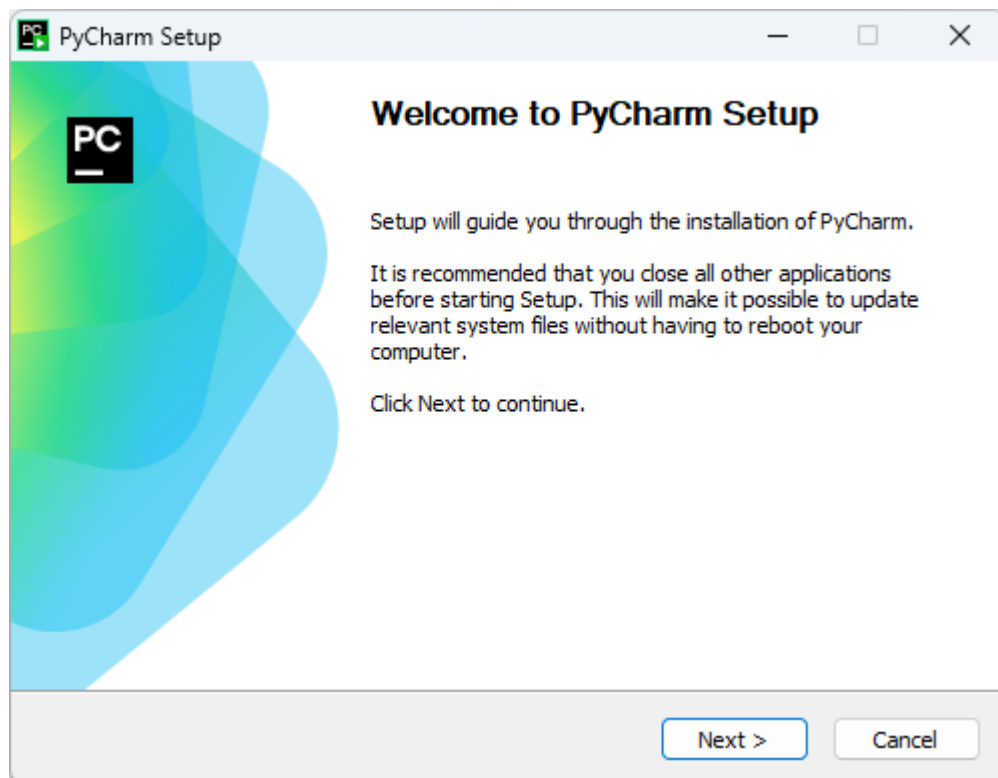
Để triển khai và chạy các ứng dụng trong đề tài, nhóm sử dụng PyCharm - một môi trường phát triển tích hợp (IDE) chuyên dụng cho ngôn ngữ Python do JetBrains phát triển. Nó hỗ trợ đầy đủ các tính năng cần thiết cho việc lập trình, quản lý thư viện, gỡ lỗi.

Cài đặt Python:

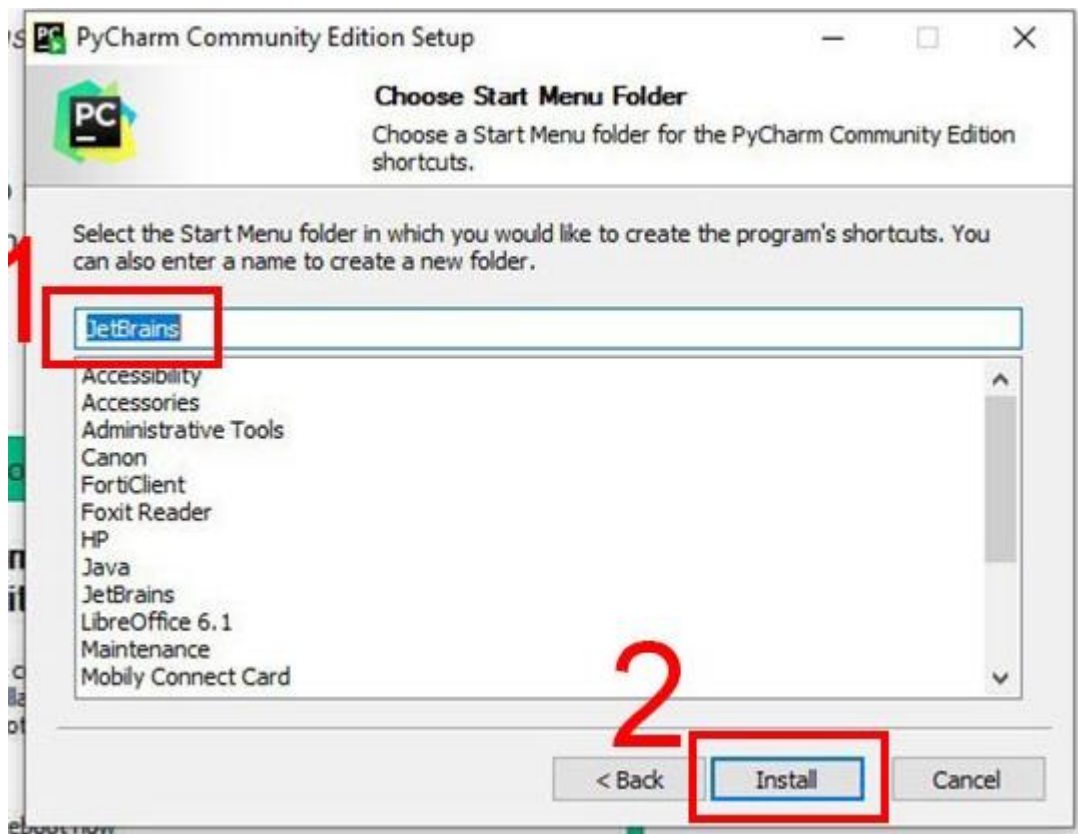
- B1: Truy cập vào trang <https://www.python.org/downloads/>
- B2: Tải và cài đặt Python phiên bản 3.10
- B3: Trong quá trình cài đặt, tích chọn Add Python to PATH

Cài đặt PyCharm:

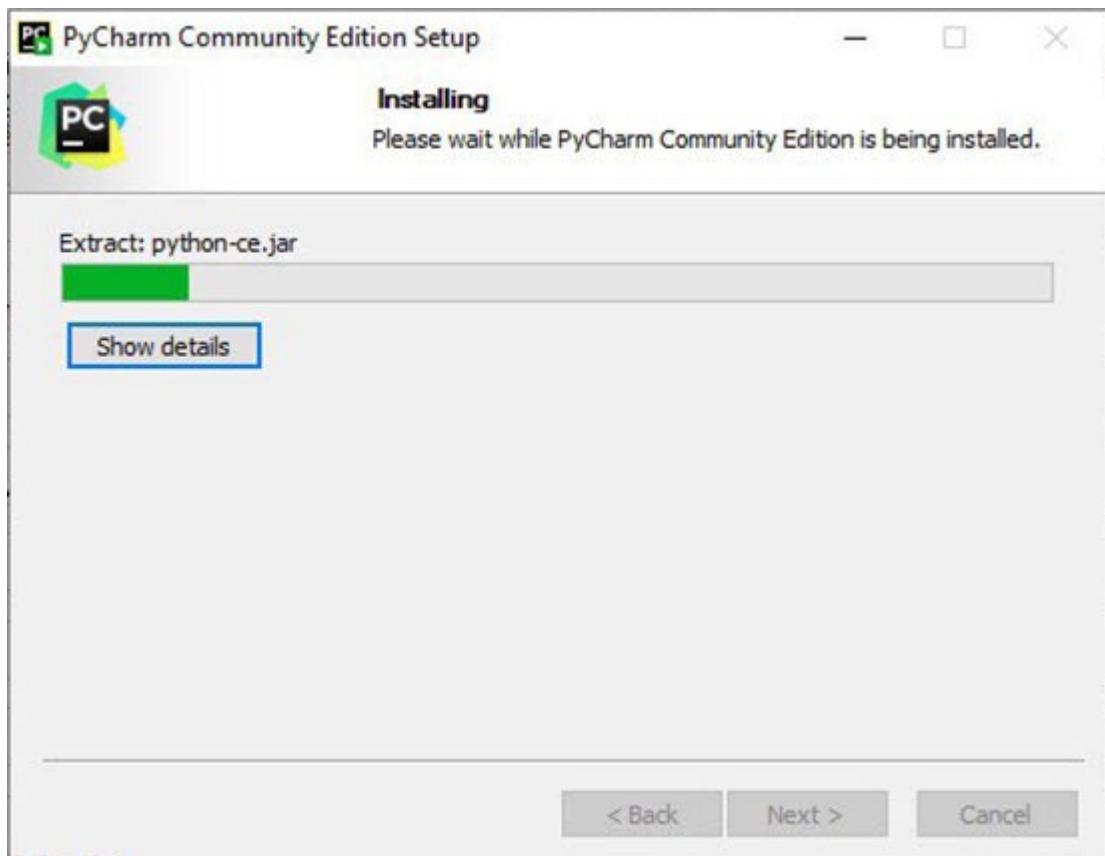
- Bước 1: Đầu tiên truy cập chủ của PyCharm, lựa chọn phiên bản phù hợp với hệ điều hành và tải xuống tệp cài đặt.
- Bước 2: Sau khi tải xuống, mở tệp lên và cài đặt:



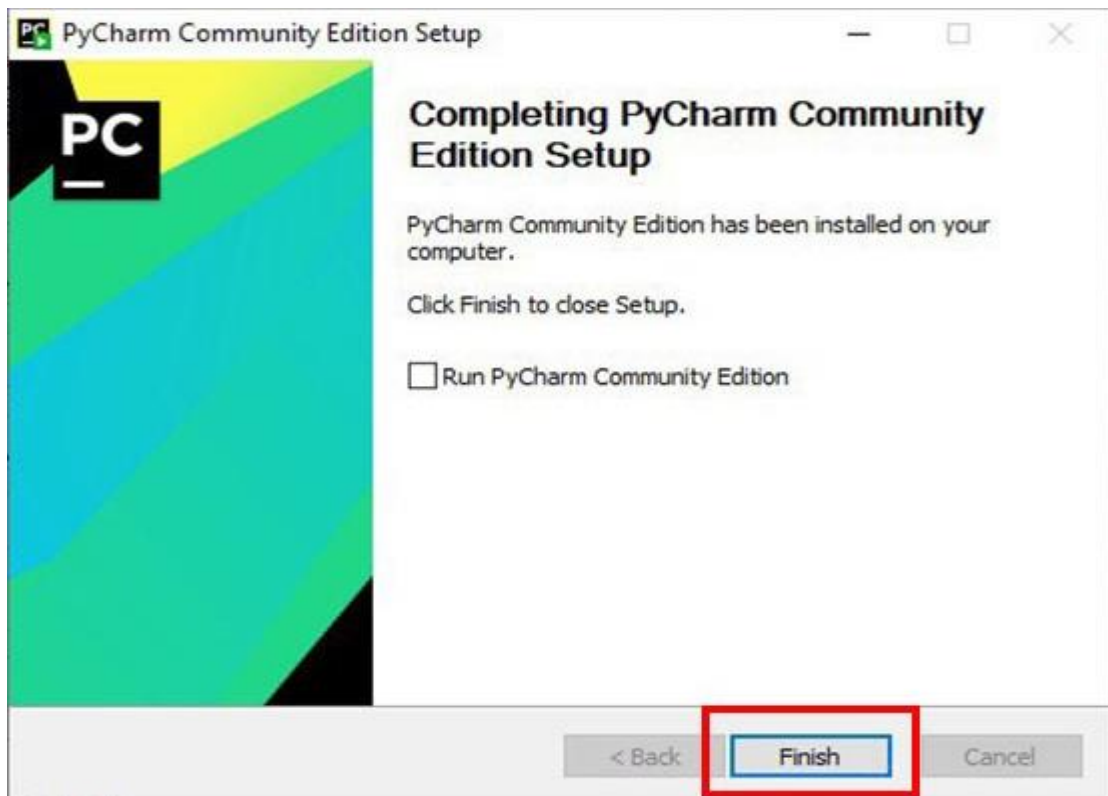
- Bước 3: Tại thanh công cụ, tùy chỉnh sang mục JetBrains, sau đó nhấn chọn Install



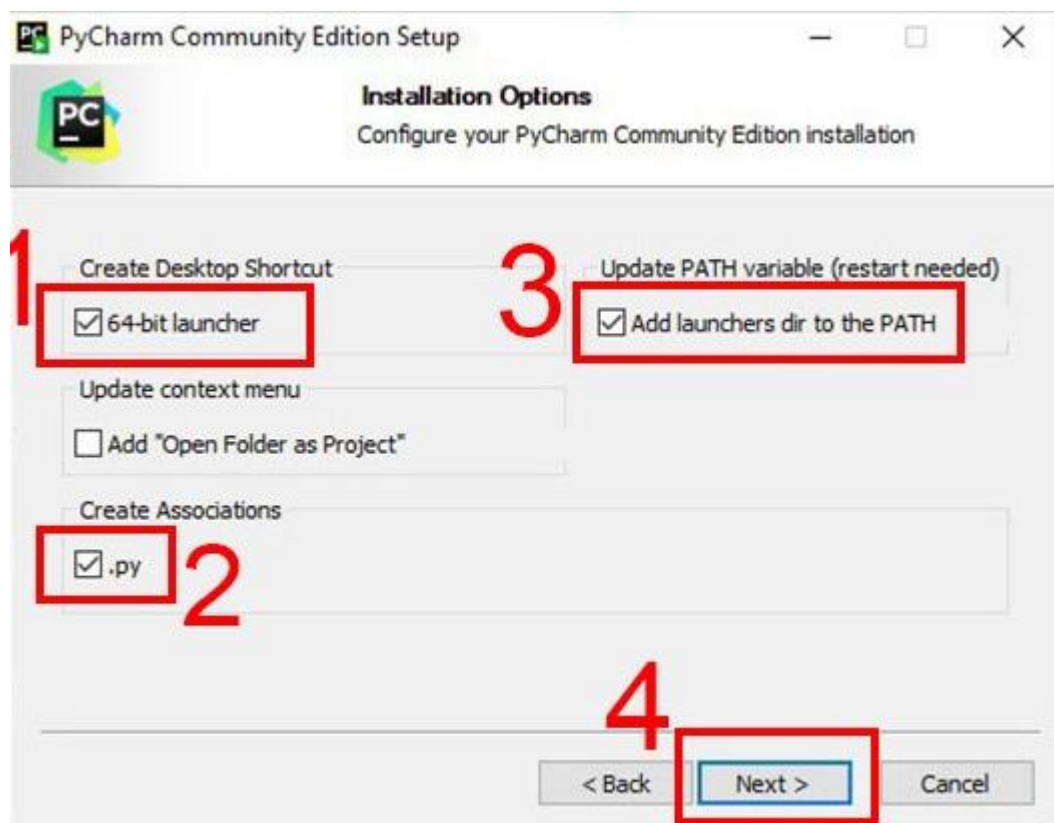
- Bước 4: Dợi quá trình Installing diễn ra



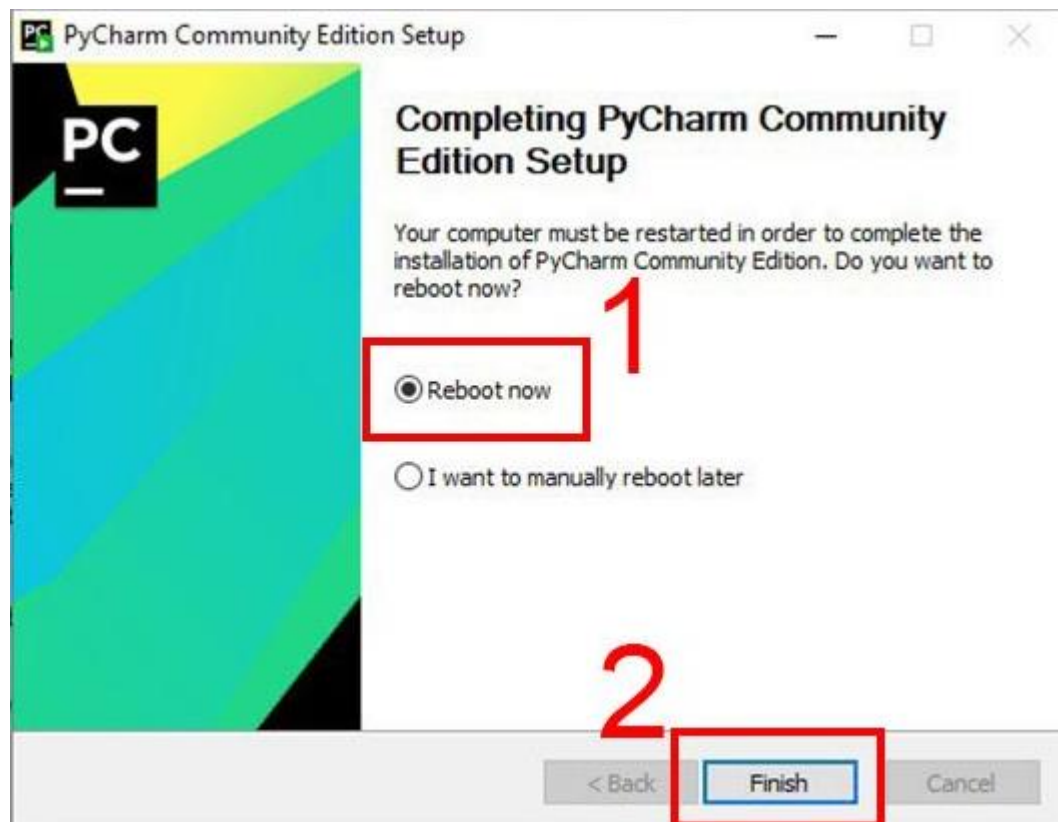
- Bước 5: Nhấn chọn Finish



- Bước 6: Tại giao diện tiếp theo, Tick vào 3 mục: 64-bit launcher, .py và Add launchers dir to the PATH nếu máy bạn chưa cài đặt Java. Sau đó nhấn Next



- Bước 7: Tick vào Reboot now, sau đó nhấn chọn Finish để hoàn thành cài đặt



3.2 Ứng dụng thực tế với “Image Blending OpenCV Python”

3.2.1 Tổng quan

Kỹ thuật pha trộn ảnh (Image Blending) là một trong những phương pháp quan trọng trong xử lý ảnh số, được ứng dụng rộng rãi trong thiết kế đồ họa, quảng cáo, chỉnh sửa ảnh và tạo hiệu ứng thị giác. Bài toán đặt ra là làm thế nào để kết hợp hai hình ảnh khác nhau thành một hình ảnh mới mà vùng tiếp giáp giữa chúng vẫn mượt mà, tự nhiên, không xuất hiện các đường cắt rõ ràng hoặc sự khác biệt đột ngột về màu sắc và chi tiết.

Trong thực tế, kỹ thuật blending được sử dụng trong nhiều tình huống:

- **Tạo ảnh quảng cáo:** kết hợp sản phẩm với một phong nền khác để đạt được tính thẩm mỹ cao.
- **Watermark/Bảo vệ bản quyền:** chèn logo hoặc chữ ký vào hình ảnh mà không làm mất nội dung chính.

- **Overlay/Hiệu ứng filter:** tạo lớp hiệu ứng ánh sáng, màu sắc hoặc texture phủ lên ảnh gốc giống các ứng dụng chỉnh sửa ảnh hiện đại.

3.2.2 Phân tích mã nguồn chương trình

a) Giới thiệu

Để minh họa rõ ràng cách thức áp dụng kỹ thuật pha trộn ảnh vào một bài toán thực tế, nhóm tiến hành xây dựng một chương trình hoàn chỉnh sử dụng Python và thư viện OpenCV. Ứng dụng này không chỉ triển khai thuật toán Gaussian Pyramid và Laplacian Pyramid ở mức lý thuyết, mà còn tích hợp giao diện đồ họa giúp người dùng thao tác trực tiếp với ảnh, qua đó thể hiện đầy đủ quy trình từ chọn dữ liệu đầu vào đến tạo ra ảnh đã blend.

Ứng dụng Image Blending OpenCV Python được xây dựng nhằm mô phỏng và minh họa kỹ thuật blending bằng Gaussian Pyramid và Laplacian Pyramid trong thư viện OpenCV. Thông qua giao diện PySimpleGUIQt, người dùng có thể:

1. Lựa chọn hai ảnh bất kỳ trên máy.
2. Xem trước ảnh được chọn.
3. Tự động thực hiện pha trộn ảnh bằng thuật toán Pyramid Blending.
4. Xem kết quả trực tiếp trong giao diện.
5. Lưu ảnh hoàn chỉnh với kích thước tùy chỉnh.

b) Import thư viện và định nghĩa hàm `blend_images`

```
# Ứng dụng Image-Blending-OpenCV-Python
import cv2 as cv
import numpy as np
import os
import tempfile
import PySimpleGUIQt as sg
```

- **cv2 (OpenCV):** xử lý ảnh, tạo Gaussian & Laplacian Pyramid, resize, blend, lưu ảnh.
- **numpy:** thao tác mảng ảnh (nối, trích xuất nửa ảnh...).

- **os:** thao tác file (đường dẫn, kiểm tra tồn tại...).
- **tempfile:** tạo file tạm phục vụ hiển thị ảnh trong GUI.
- **PySimpleGUIQt:** xây dựng giao diện người dùng, thay thế Tkinter, tránh lỗi Tcl/Tk.

```
def blend_images(path1, path2, levels=6, mode="Trái-Phải"): 1 usage
    img1 = cv.imread(path1)
    img2 = cv.imread(path2)

    if img1 is None or img2 is None:
        raise Exception("Không thể đọc ảnh.")

    if img1.shape != img2.shape:
        img2 = cv.resize(img2, dsize=(img1.shape[1], img1.shape[0]))
```

- Hàm nhận vào đường dẫn 2 ảnh (path1, path2), đồng thời hỗ trợ các tham số levels (số mức của pyramid) và mode (kiểu ghép ảnh).
- Đọc ảnh bằng cv.imread.
- Nếu không đọc được ảnh, báo lỗi.
- Nếu ảnh 2 khác kích thước ảnh 1, resize để trộn được.

```
gp1 = [img1.copy()]
gp2 = [img2.copy()]
for i in range(levels):
    gp1.append(cv.pyrDown(gp1[-1]))
    gp2.append(cv.pyrDown(gp2[-1]))
```

- Tạo Gaussian Pyramid cho hai ảnh.
- pyrDown giảm độ phân giải ảnh từng cấp.
- levels là số cấp giảm dần (mặc định 6).

```

lp1 = [gp1[-1]]
lp2 = [gp2[-1]]

for i in range(levels - 1, -1, -1):
    up1 = cv.pyrUp(gp1[i + 1])
    up1 = cv.resize(up1, dsize: (gp1[i].shape[1], gp1[i].shape[0]))
    lp1.append(cv.subtract(gp1[i], up1))

    up2 = cv.pyrUp(gp2[i + 1])
    up2 = cv.resize(up2, dsize: (gp2[i].shape[1], gp2[i].shape[0]))
    lp2.append(cv.subtract(gp2[i], up2))

```

- Tạo Laplacian Pyramid từ Gaussian Pyramid.
- pyrUp phóng ảnh cấp cao lên cấp thấp.
- subtract để lấy phần chi tiết (khác biệt) của ảnh.
- Kết quả là mỗi cấp ảnh lưu chi tiết của ảnh gốc.

```

LS = []
for la, lb in zip(lp1, lp2):
    cols = la.shape[1]
    rows = la.shape[0]

    if mode == "Trái-Phải":
        LS.append(np.hstack((la[:, :cols//2], lb[:, cols//2:])))
    else:
        # Trên - Dưới
        LS.append(np.vstack((la[:rows//2, :], lb[rows//2:, :])))

```

- Danh sách LS dùng để lưu các mức ảnh đã được trộn từ hai Laplacian pyramid lp1 và lp2.
- Với mỗi cặp mức Laplacian la (từ ảnh 1) và lb (từ ảnh 2), chương trình lấy kích thước của ảnh để xác định điểm chia.
- Ở chế độ **Trái – Phải**, ảnh kết quả ở mỗi mức được ghép bằng cách:
 - + Lấy nửa **bên trái** của mức Laplacian từ ảnh 1 (la[:, :cols//2]).
 - + Lấy nửa **bên phải** của mức Laplacian từ ảnh 2 (lb[:, cols//2:]).
 - + Dùng np.hstack để ghép hai phần thành một ảnh hoàn chỉnh.
- Khi chọn chế độ **Trên – Dưới**, phép ghép tương tự nhưng theo chiều dọc:

- + Lấy nửa trên từ ảnh 1.
- + Lấy nửa dưới từ ảnh 2.
- Dùng np.vstack để nối theo chiều dọc.
- Việc thực hiện ghép như vậy ở **tất cả các cấp của Laplacian pyramid** giúp tạo ra sự chuyển tiếp mượt mà giữa hai ảnh, tránh đường chia bị gãy hoặc lộ ranh giới khi ghép trực tiếp ở ảnh gốc.

```
result = LS[0]
for i in range(1, len(LS)):
    result = cv.pyrUp(result)
    result = cv.resize(result, dsize: (LS[i].shape[1], LS[i].shape[0]))
    result = cv.add(result, LS[i])

return result
```

- Tái tạo ảnh cuối cùng từ Laplacian Pyramid đã ghép.
- pyrUp và add để phóng ảnh và cộng dồn các chi tiết.
- Trả về ảnh trộn mượt mà.

c) Lưu file tạm và xem trước ảnh

```
def save_temp(temp_img): 2 usages
    file = tempfile.mktemp(suffix=".png")
    cv.imwrite(file, temp_img)
    return file
```

- Lưu ảnh tạm vào file .png.
- Trả về đường dẫn tạm để GUI hiển thị.

```
def preview(path, w=350, h=250): 2 usages
    img = cv.imread(path)
    if img is None:
        return None
    img = cv.resize(img, dsize: (w, h))
    return save_temp(img)
```

- Đọc ảnh và resize nhỏ để hiển thị nhanh trong GUI.
- Trả về đường dẫn ảnh tạm.

d) Tạo giao diện GUI

```

def make_gui(): 1 usage
    sg.theme("LightGrey1")

    layout = [
        [sg.Text( text: "IMAGE BLENDING", font=("Arial",16), justification="center")],

        [sg.Text("Ảnh 1:"), sg.Input(key="-F1-", size=(30,1)), sg.FileBrowse("Chọn")],
        [sg.Image(key="-IMG1-")],

        [sg.Text("Ảnh 2:"), sg.Input(key="-F2-", size=(30,1)), sg.FileBrowse("Chọn")],
        [sg.Image(key="-IMG2-")],

        [sg.Text( text: "Kiểu ghép:", size=(12, 1)),
         sg.Combo( values=["Trái-Phải", "Trên-Dưới"], default_value="Trái-Phải", key="-MODE-", size=(12, 1))],

        [sg.Button( button_text: "Kết Hợp", key="-RUN-", size=(12,1)),
         sg.Button( button_text: "Lưu Ảnh", key="-SAVE-", size=(12,1), disabled=True),
         sg.Button( button_text: "Thoát", size=(12,1))],

        [sg.Text("Kết quả:"),
         [sg.Image(key="-OUT-", size=(450,450))],

        [sg.Text( text: "",key="-STATUS-", size=(40,1))]

    ]

    return sg.Window( title: "Image Blend", layout, finalize=True)

```

- Tạo GUI gồm: chọn file 2 ảnh, hiển thị ảnh, nút "Kết Hợp", nút “Kiểu ghép”, nút "Lưu Ảnh", nút “Thoát”.
- Hiển thị kết quả trộn và trạng thái xử lý.

e) Popup lưu ảnh với kích thước tùy chọn

```

def popup_save_with_size(result): 1 usage
    layout = [
        [sg.Text( text: "Chọn nơi lưu ảnh (ten.png/jpg)", font=("Arial", 12))],
        [sg.Input(key="-SAVE_PATH-", size=(40, 1)),
         sg.FileSaveAs( button_text: "Chọn file", file_types=(("PNG Image", "*.png")),)],

        [
            sg.Text( text: "Chiều rộng", size=(10, 1), justification="right", pad=(0, 7)),
            sg.Input( default_text: "450", key="-W-", size=(9, 0.55), pad=((5, 30), 0)),

            sg.Text( text: "Chiều dài", size=(10, 1), justification="right", pad=(0, 7)),
            sg.Input( default_text: "450", key="-H-", size=(6.5, 0.7))

        ],

        [sg.Button( button_text: "OK", size=(12, 1), button_color=("white", "green")),
         sg.Button( button_text: "Hủy", size=(12, 1), button_color=("white", "red"))]

    ]

    popup = sg.Window( title: "Lưu File", layout, finalize=True)

```

```

while True:
    ev, val = popup.read()
    if ev in (None, "Hủy"):
        popup.close()
        return None
    if ev == "OK" and val["-SAVE_PATH-"]:
        try:
            w = int(val["-W-"])
            h = int(val["-H-"])
            result_resized = cv.resize(result, dsize=(w, h), interpolation=cv.INTER_AREA)
            cv.imwrite(val["-SAVE_PATH-"], result_resized)
            popup.close()
            return val["-SAVE_PATH-"]
        except:
            sg.popup("⚠ Kích thước không hợp lệ! Vui lòng nhập số nguyên.")

```

- Popup cho người dùng nhập tên file và kích thước ảnh muốn lưu.
- Resize ảnh kết quả về kích thước nhập trước khi lưu.
- Kiểm tra lỗi nếu nhập sai.

f) Hàm main điều khiển GUI

```

def main(): 1 usage
    win = make_gui()
    result = None

    while True:
        e, v = win.read()
        if e in (None, "Thoát"):
            break

        if e == "-F1-":
            if v["-F1-"]:
                prev = preview(v["-F1-"])
                if prev: win["-IMG1-"].update(filename=prev)

        if e == "-F2-":
            if v["-F2-"]:
                prev = preview(v["-F2-"])
                if prev: win["-IMG2-"].update(filename=prev)

```

```

if e == "-RUN-":
    if not v["-F1-"] or not v["-F2-"]:
        win["-STATUS-"].update("▲ Hãy chọn cả 2 ảnh!")
        continue

    mode = v["-MODE-"]
    win["-STATUS-"].update("Đang xử lý...")

    result = blend_images(v["-F1-"], v["-F2-"], mode=mode)

    result_resized = cv.resize(result, dsize: (450, 450), interpolation=cv.INTER_AREA)
    temp = save_temp(result_resized)

    win["-OUT-"].update(filename=temp)
    win["-SAVE-"].update(disabled=False)
    win["-STATUS-"].update("✓ Hoàn tất!")

if e == "-SAVE-" and result is not None:
    saved_path = popup_save_with_size(result)
    if saved_path:
        win["-STATUS-"].update(f"✓ Đã lưu: {saved_path}")

win.close()

```

- Vòng lặp xử lý các sự kiện GUI.
- Khi chọn ảnh, hiển thị preview.
- Ghép ảnh theo mode (Ghép ngang hoặc dọc).
- Khi nhấn "Kết Hợp", gọi `blend_images` để trộn 2 ảnh, hiển thị kết quả và mở nút "Lưu Ảnh".
- Khi nhấn "Lưu Ảnh", mở popup để lưu với kích thước tùy chọn.

g) Chạy chương trình

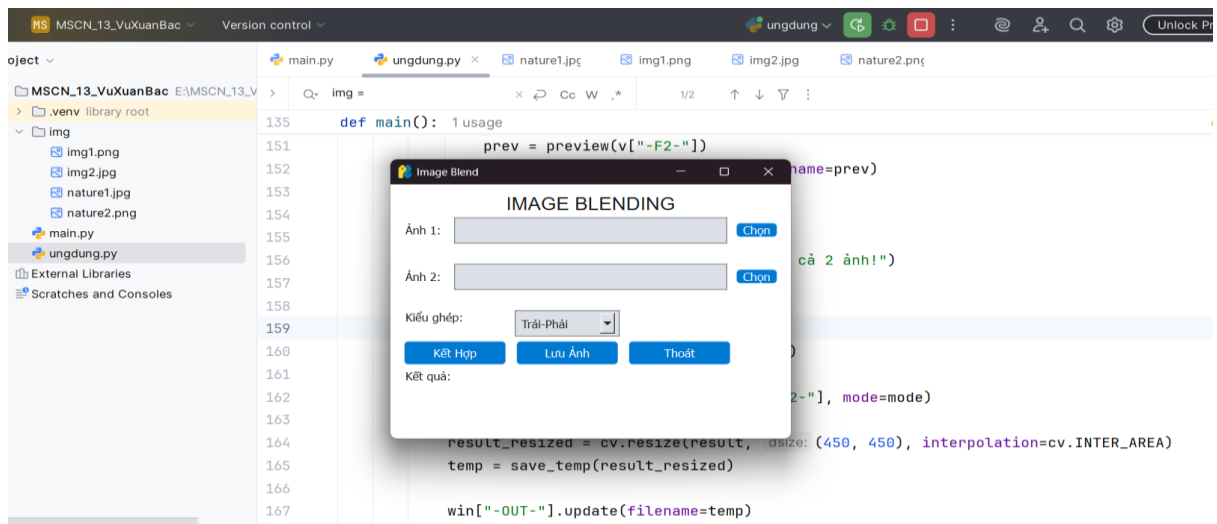
```

▶ if __name__ == "__main__":
    main()

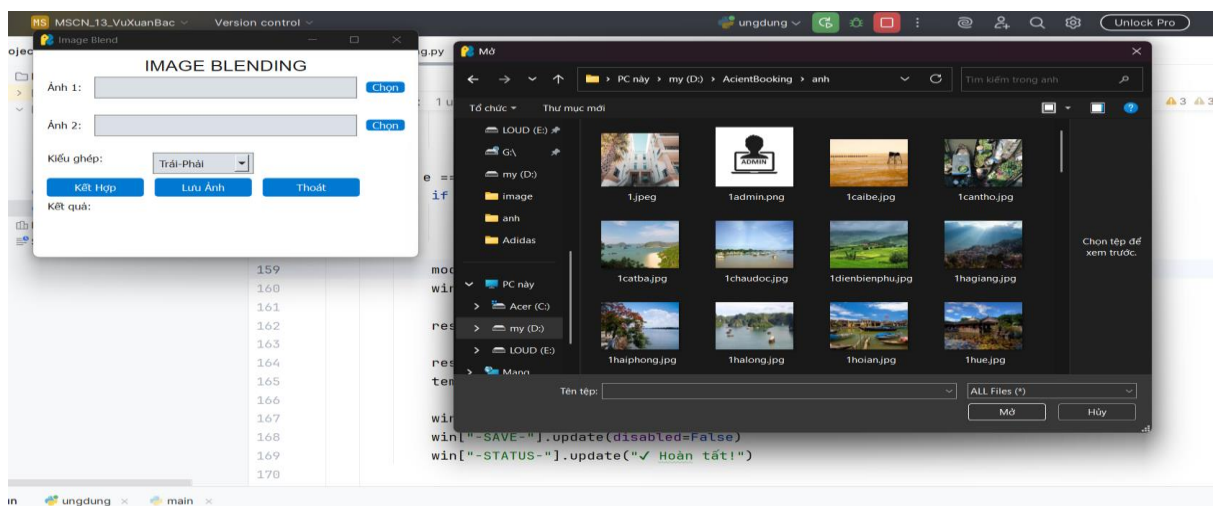
```

3.2.3 Kết quả và đánh giá

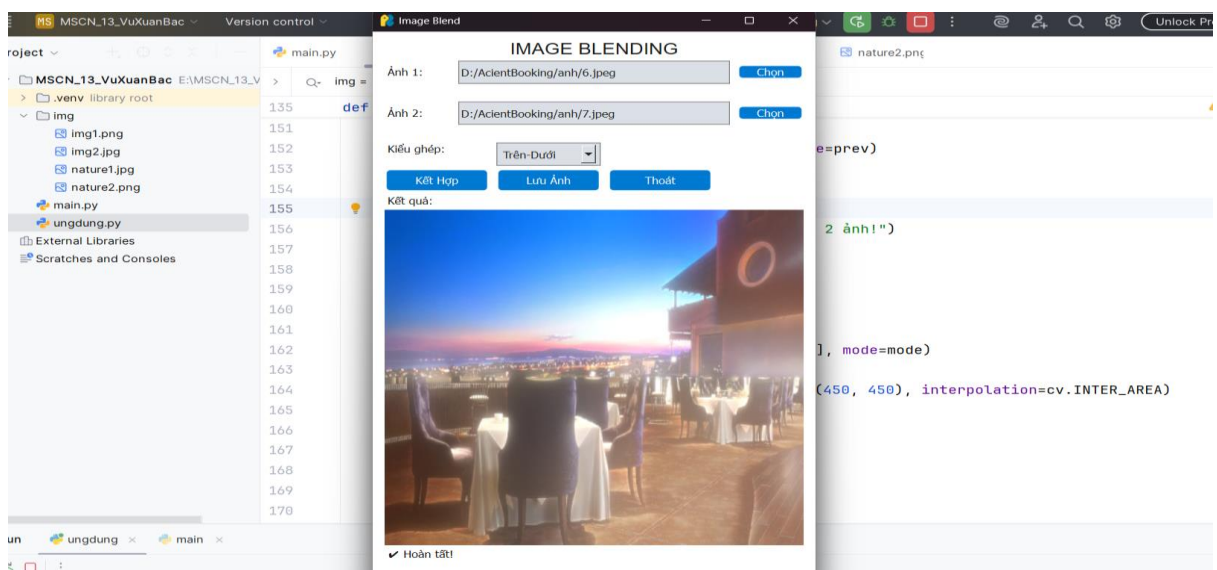
a) Kết quả



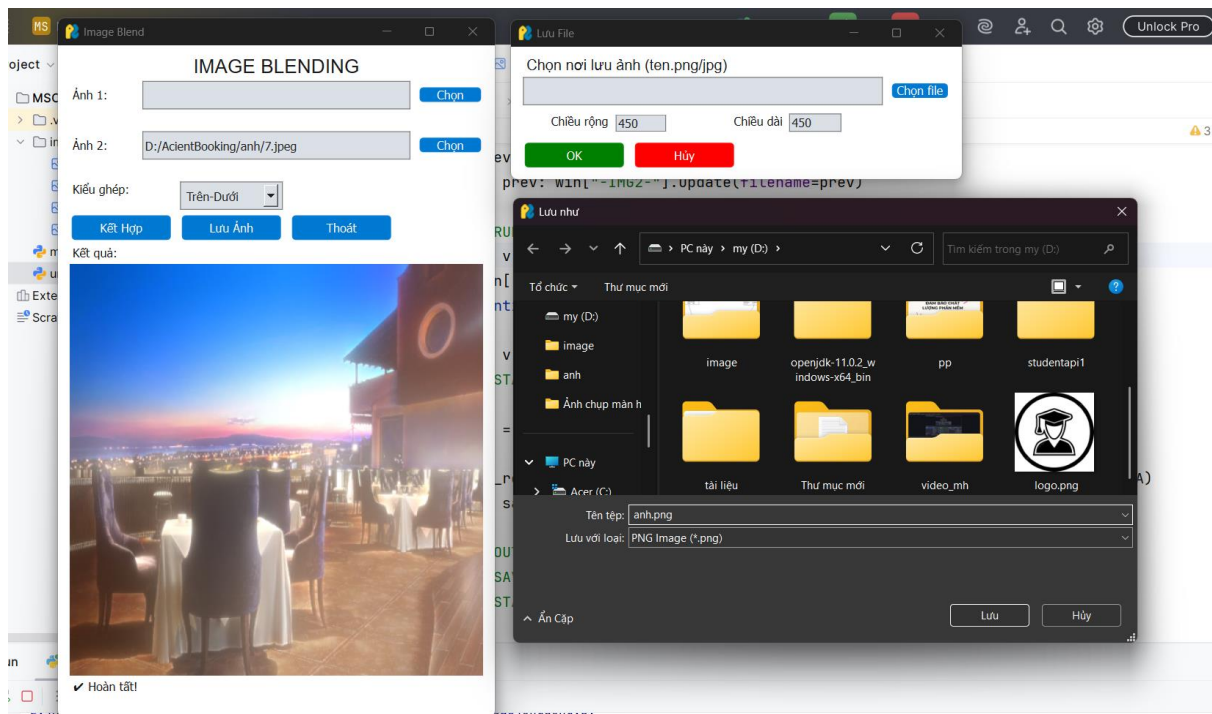
Hình 3.1. Giao diện chính



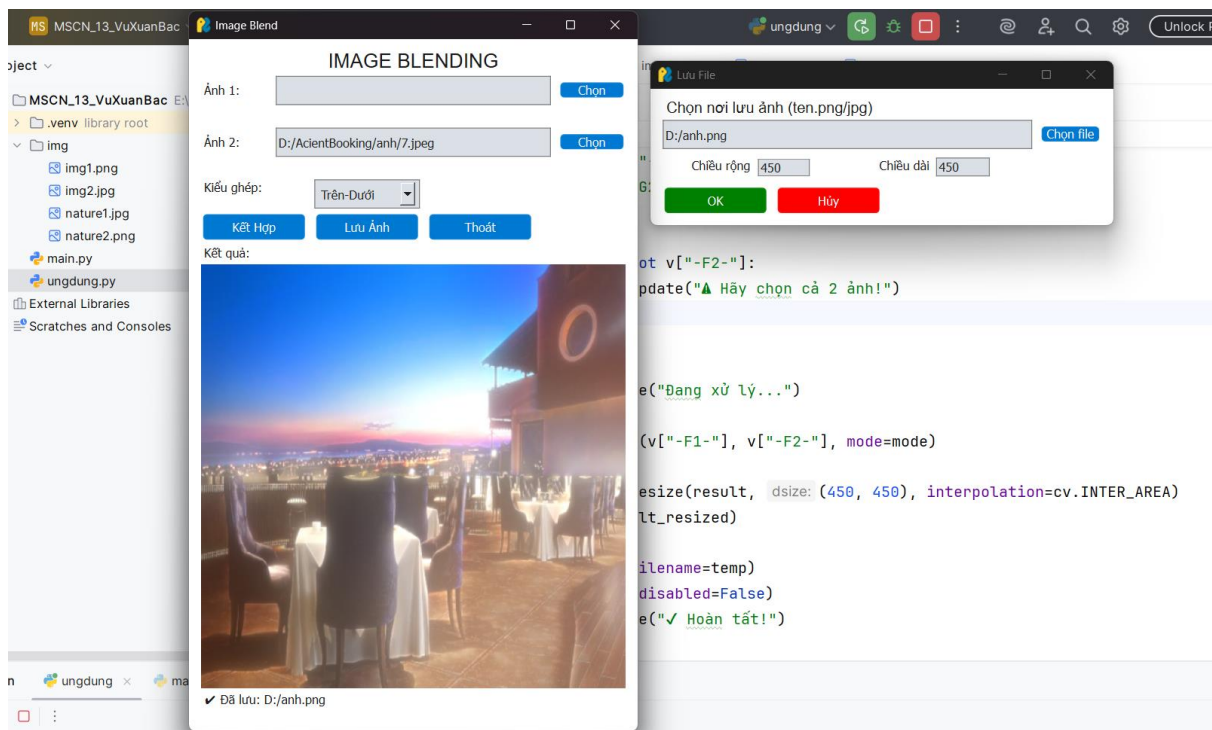
Hình 3.2. Giao diện chọn ảnh



Hình 3.3. Giao diện kết quả trộn ảnh



Hình 3.4. Giao diện lưu ảnh



Hình 3.5. Giao diện lưu ảnh thành công

b) Đánh giá

Ưu điểm:

- Kỹ thuật xử lý ảnh chuyên nghiệp:

- + Sử dụng Gaussian Pyramid và Laplacian Pyramid giúp kết hợp ảnh mượt, không bị lộ đường ghép.
- + Kết quả đẹp hơn nhiều so với ghép thủ công (hstack).
- Giao diện đơn giản, dễ sử dụng:
 - + Chọn ảnh, xem trước, chọn kiểu trộn, trộn ảnh, lưu ảnh đều trực quan.
 - + Không cần dùng dòng lệnh, phù hợp người dùng phổ thông.
- Có khả năng tùy chỉnh khi lưu ảnh:
 - + Người dùng chọn kích thước ảnh mong muốn.
 - + Ảnh tự động resize khi lưu.
- Hoạt động ổn định:
 - + Chương trình kiểm tra lỗi ảnh không tồn tại hoặc sai đường dẫn.
 - + Không bị crash khi xử lý ảnh lớn.

Hạn chế:

- Tốc độ xử lý phụ thuộc cấu hình máy:
 - + Với ảnh độ phân giải rất lớn, tạo pyramid nhiều tầng có thể chậm.

TÀI LIỆU THAM KHẢO

- [1] Trường Đại học Công nghiệp Hà Nội (2024), Giáo trình một số công nghệ phần mềm. Nhà xuất bản Thống kê
- [2] Lê Văn Phùng (2021). Python và OpenCV: Từ cơ bản đến nâng cao trong xử lý ảnh. Nhà xuất bản Thông tin và Truyền thông
- [3] Nguyễn Văn Hiệp (2020). Xử lý ảnh và thị giác máy tính với OpenCV và Python. Nhà xuất bản Khoa học và Kỹ thuật